SAP University Competence Center
Chair for Information Systems and Business Process Management
TUM School of Computation, Information, and Technology
Technical University of Munich

# Advanced Practical Course – Next Generation Data Center Operations for SAP Enterprise Software (IN2128, IN2106, IN212810)

## Containers

**Simon Fuchs, Maximilian Niedermeier, Noah Kim, Leon Kist**

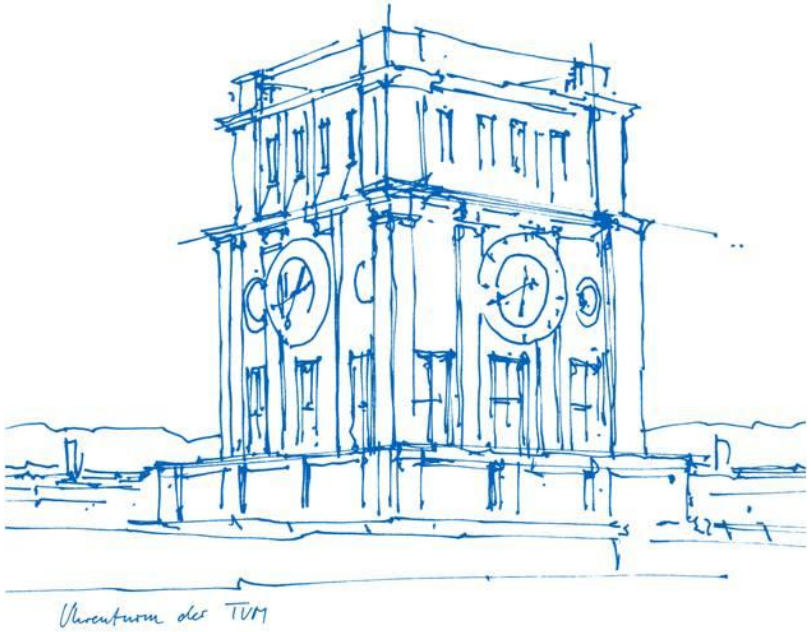
SAP University Competence Center

Information Systems and Business Process Management

TUM School of Computation, Information and Technology

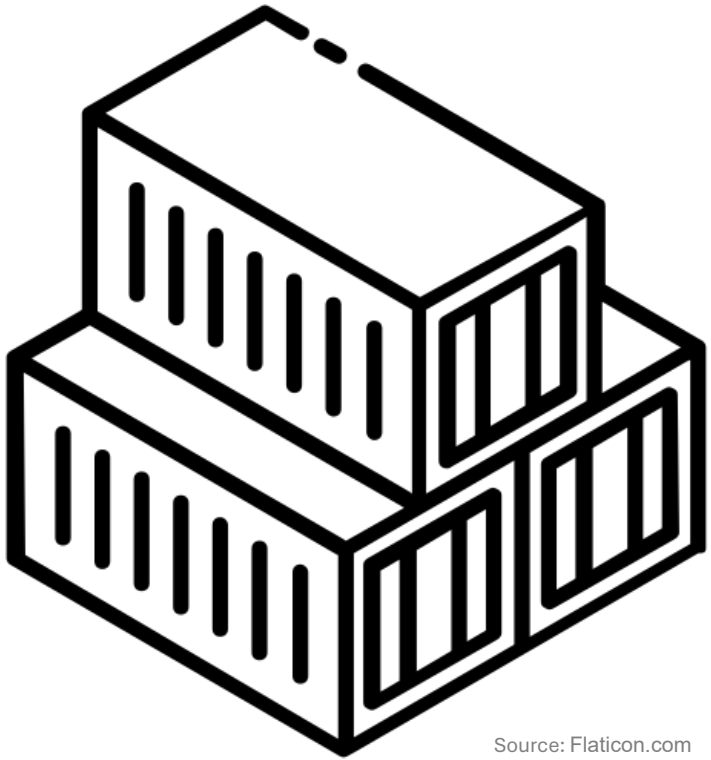
Technical University of Munich

17.04.2026



# Containers

# Introduction



Source: Flaticon.com

# Introduction

Outline

- Containers in General
- Container Environment
- Container Orchestration

# Introduction

Outline

- Containers in General → **What is a container?**
  - Definition
  - Container vs VM
- Container Environment
- Container Orchestration

# Introduction

Outline

- Containers in General
  - Definition
  - Container vs VM
- Container Environment → **How do containers run?**
  - Container Registries & Building Images
  - Running Containers
  - Docker & Podman
- Container Orchestration

# Introduction

Outline

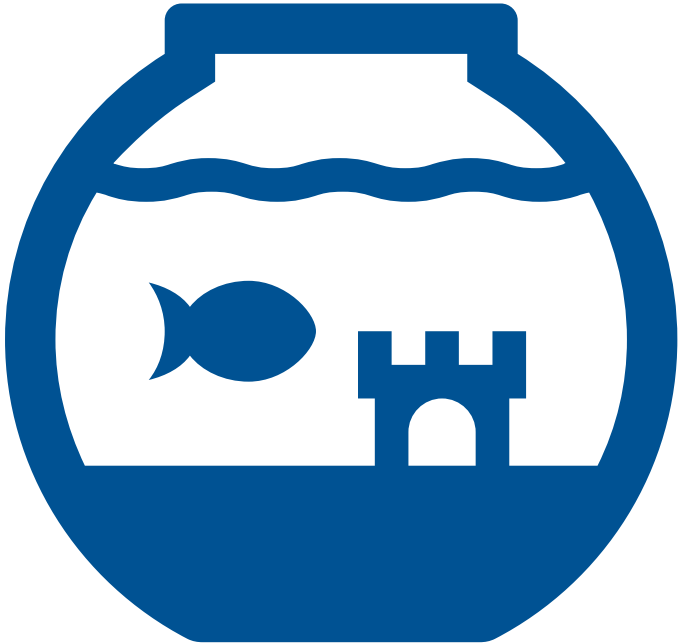
- Containers in General
  - Definition
  - Container vs VM
- Container Environment
  - Container Registries
  - Building and Running Containers
  - Docker & Podman
- Container Orchestration → **How are containers scaled? (Monday)**
  - Hyperscaler & Cloud
  - Kubernetes

# Introduction

Group exercise:

- Form groups of 3-5 people → Your table
- As a group: Think about isolated environments (10 minutes)
  - Come up with 2 isolated environments **outside of computing**
  - Answer:
    - **What** is the environment?
    - **Why** is it isolated?
    - **How** is it isolated, i.e., what mechanism is used?
- Present your results

# Introduction



Group exercise:

- Example: Fishbowl
  - **What** is the environment?
    - Aquatic landscape
  - **Why** is it isolated?
    - Allows water-dependent species (e.g., fish) to live outside of natural bodies of water
  - **How** is it isolated, i.e., what mechanism is used?
    - Glass bowl holds water, filters create specific conditions

# Containers in General

What is a container?

- According to Docker:
  - Container: “[…] a runnable instance of an **image**”

# Containers in General

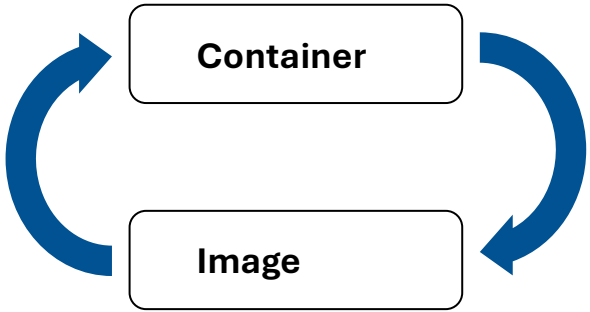
What is a container?

- According to Docker:
  - Container: “[…] a runnable instance of an **image**”
  - Image: “[…] template with instructions for creating a […] **container**”

# Containers in General

What is a container?

- According to Docker:



# Containers in General | Images

What is a container image?

- **Archive**:
  - Configuration files → describe the container (e.g, components, filesystem layout, …)
  - All external files required to build the container i.e., to run the desired application

- Popular format → specified by the **Open Container Initiative**

# Containers in General | Images

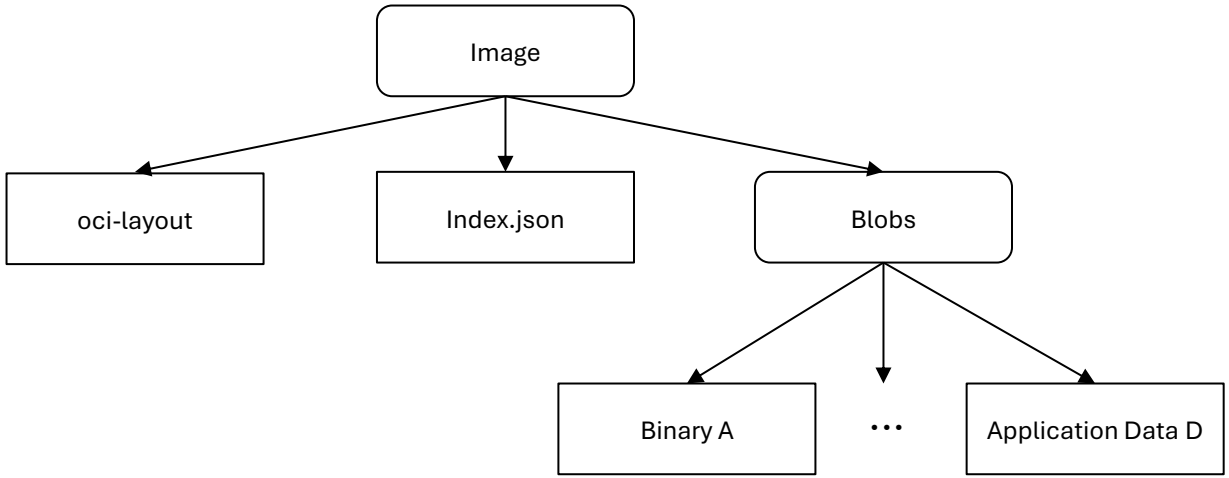
Open Container Initiative (OCI):

- Established in 2015 by Docker and other groups in the container industry
- Open governance structure
- Creating **open industry standards** around container formats and runtimes
- Currently:  Runtime Specification, Image Specification, Distribution Specification

# Containers in General | Images

**OCI Image** format:

- Layout:



# Containers in General | Images

**OCI Image** format:

- Layout:
  - *oci-layout* file
    - Base of an OCI Layout
  - *index.json* file
    - Annotated list of manifests
    - Entrypoint for image
  - *blobs* directory
    - All necessary files (for the container), e.g., binaries, application data, …

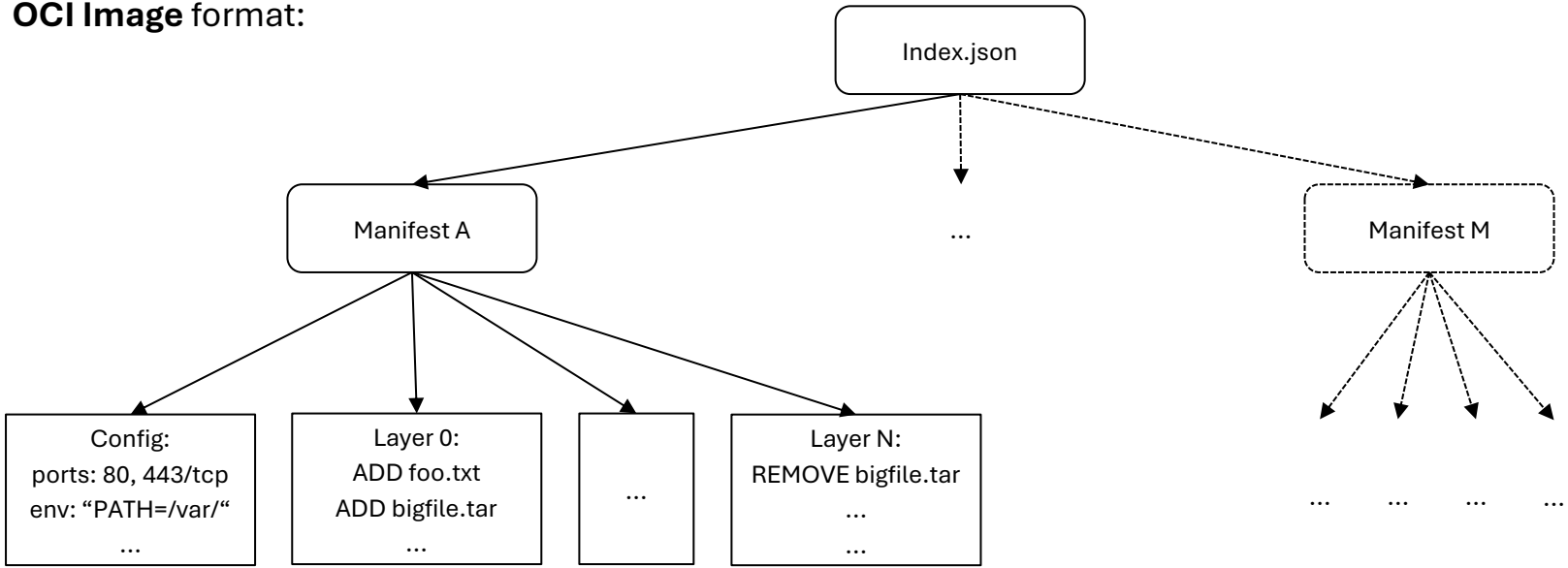
# Containers in General | Images

**OCI Image** format:

- Manifest:
  - Describes components of a container
  - Specific architecture and operating system
  - Contains:
    - *Configuration* → properties of the container (e.g., open ports, environment variables, **entrypoint**, …)
    - *Set of layers*
      - *Layer* → Series of file system changes (Additions, Modifications, Removals)

# Containers in General | Images

**OCI Image** format:



# Containers in General | Images

**OCI Image** format:

- Demo:

**alpine** Docker Official Image · 1B+ · 10K+

A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size!

OPERATING SYSTEMS

Source: https://hub.docker.com/_/alpine/

# Containers in General | Containers

What is a container?

- Initial definition: “[…] a runnable instance of an **image**”
- Image → defines a very specific system state (including running process)

**Container:** runnable instance of specific system state

→ Requires an **isolated** environment (file system, environment variables, ports) within another system

# Containers in General | Containers

How can we create such an enclosed environment?

## Containers in General | Containers

How can we create such an enclosed environment?

# Linux Namespaces

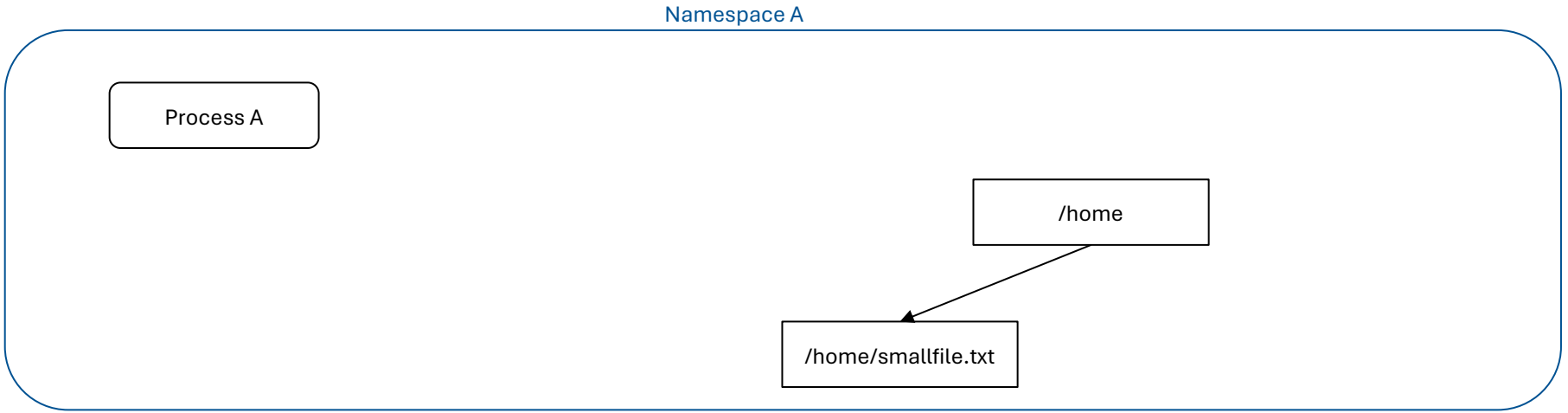
# Containers in General | Linux Namespaces

**Linux Namespaces:**

- Part of the Linux kernel
- Wraps global system resources (filesystem, network, etc.):
  - Appear as an isolated instance in each namespace
  - Changes are invisible in other namespaces
- Processes become members of namespaces on creation

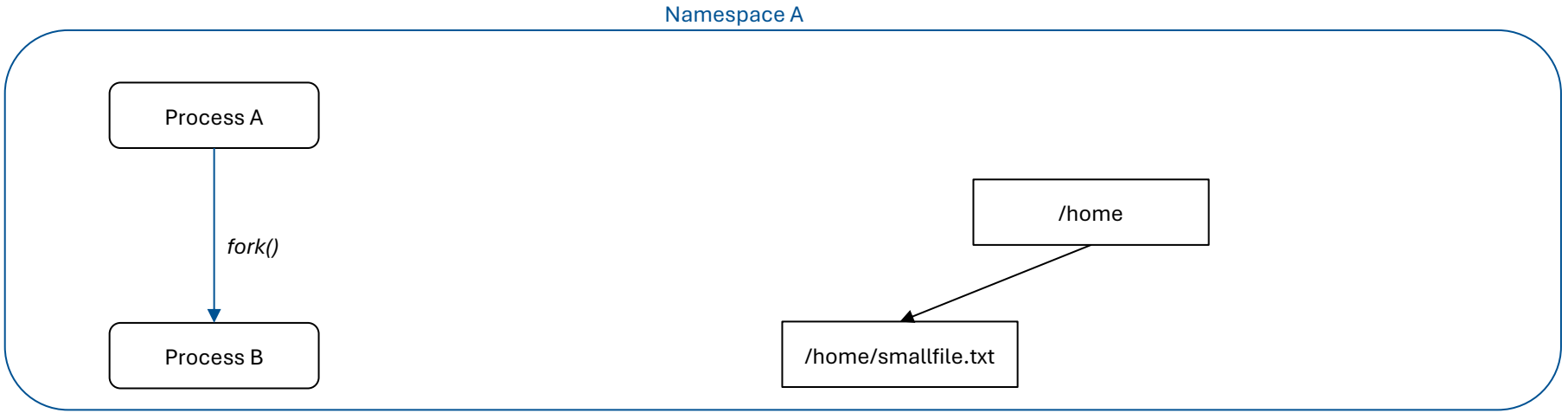
# Containers in General | Linux Namespaces

## Example: Same Namespace



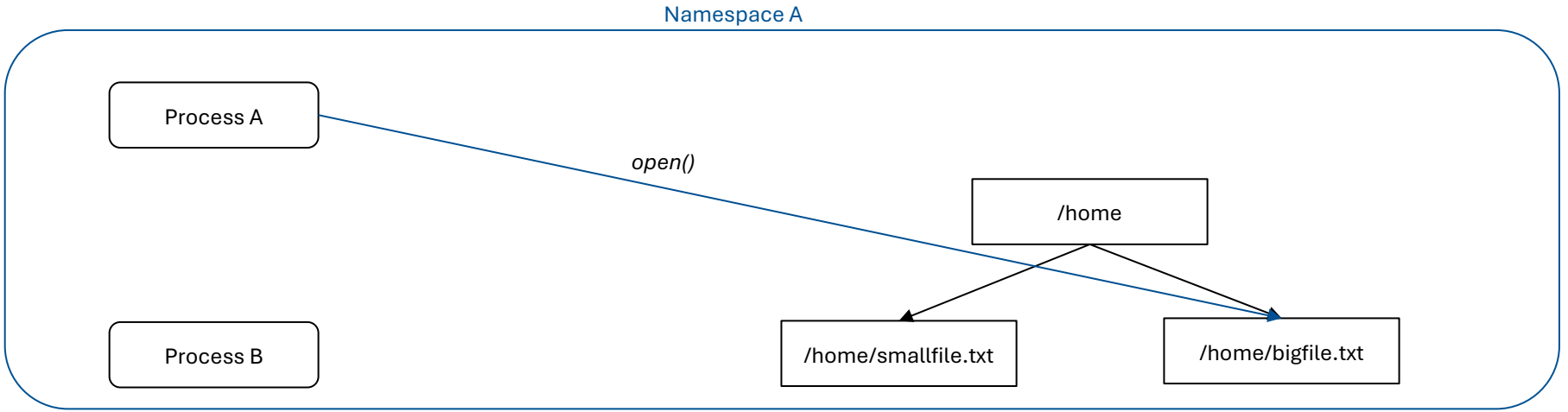
# Containers in General | Linux Namespaces

## Example: Same Namespace



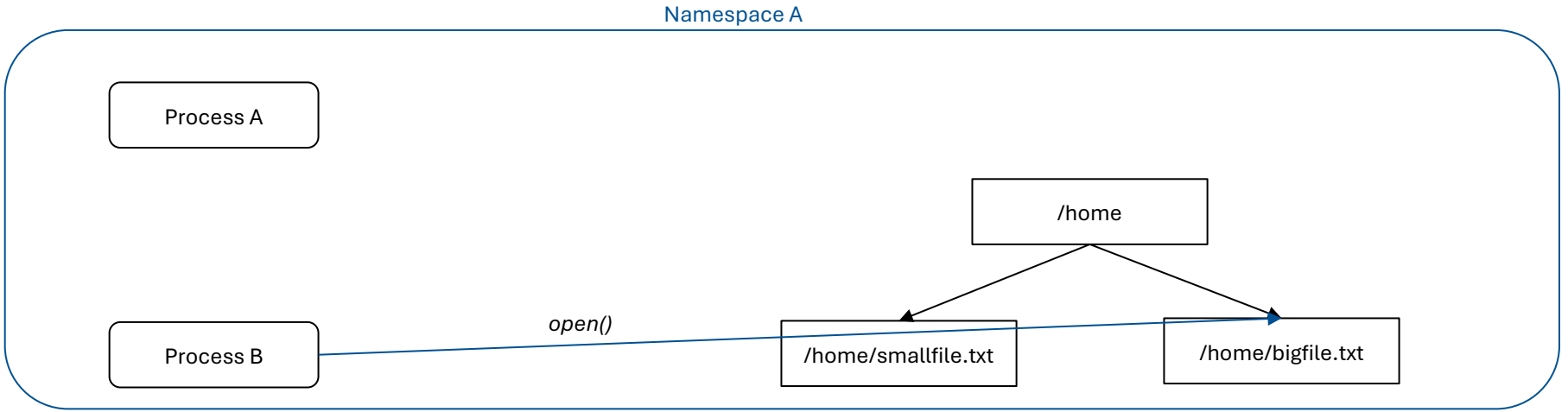
# Containers in General | Linux Namespaces

## Example: Same Namespace



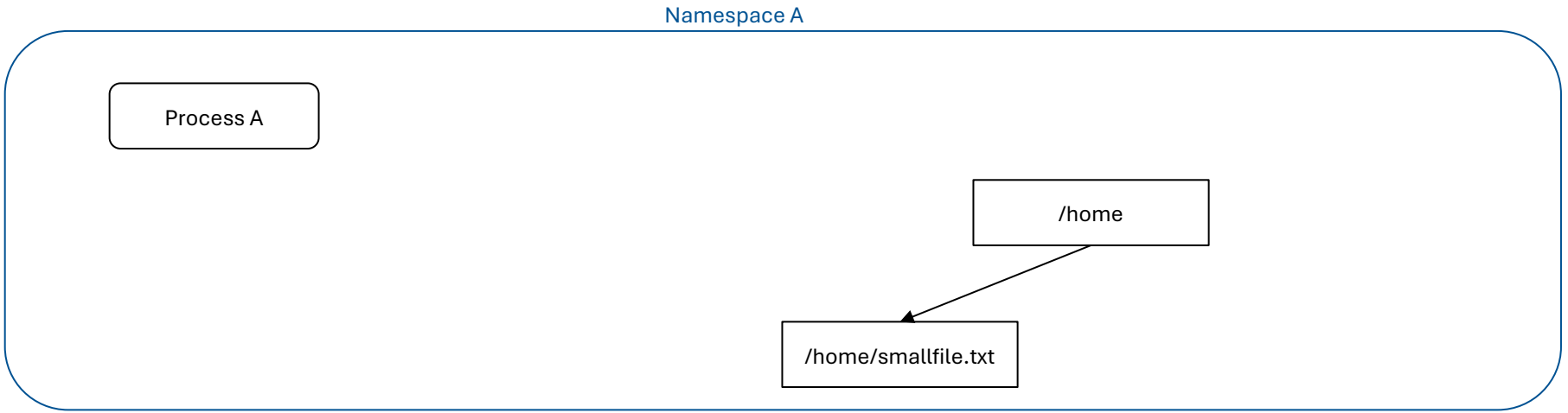
# Containers in General | Linux Namespaces

## Example: Same Namespace



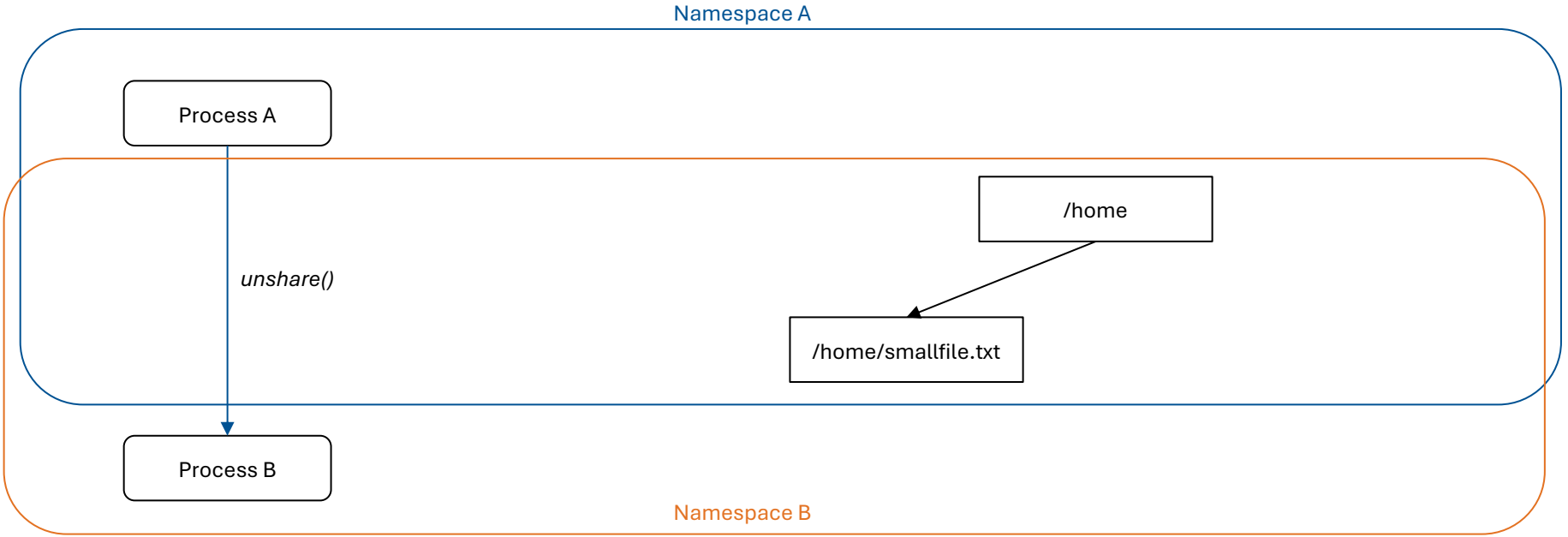
# Containers in General | Linux Namespaces

## Example: Different Namespaces



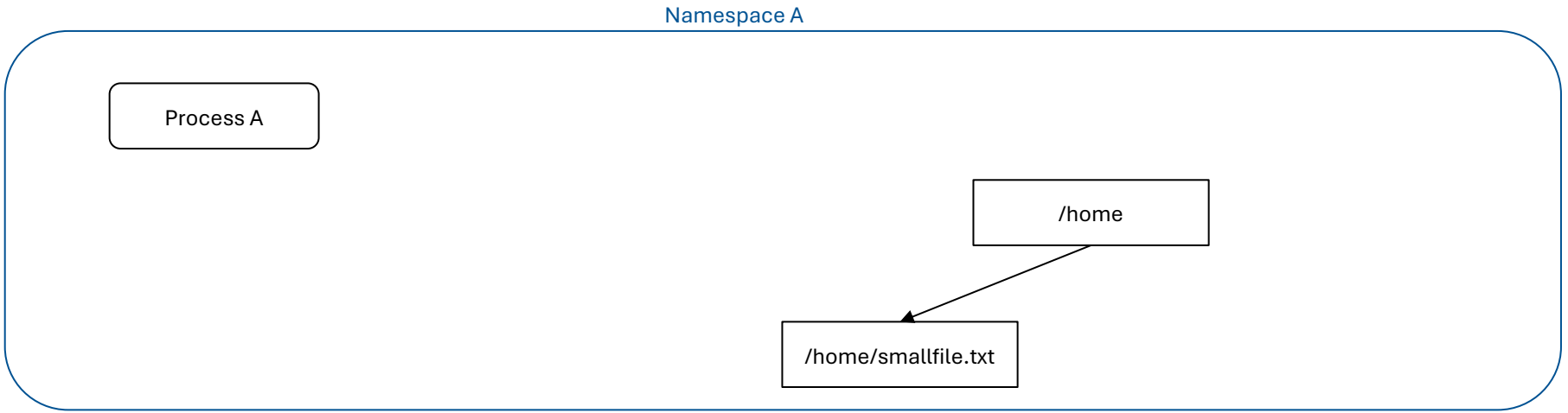
# Containers in General | Linux Namespaces

## Example: Different Namespaces



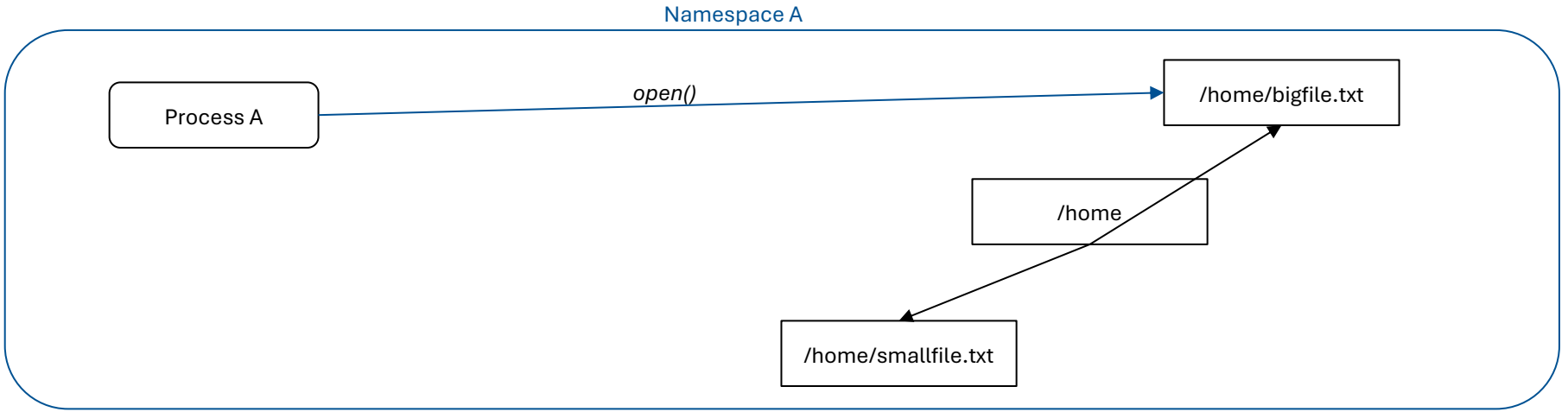
# Containers in General | Linux Namespaces

## Example: Different Namespaces



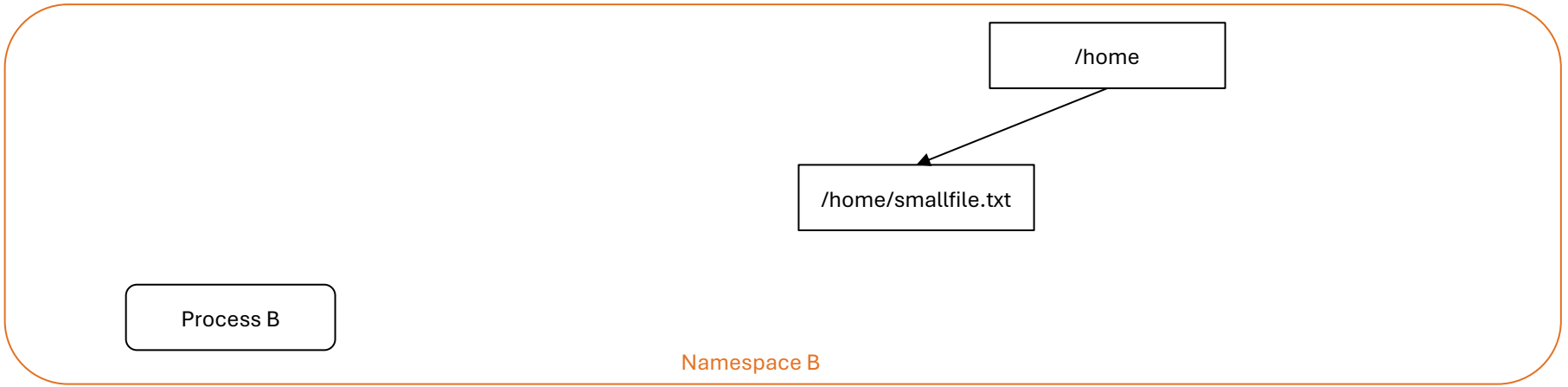
# Containers in General | Linux Namespaces

## Example: Different Namespaces



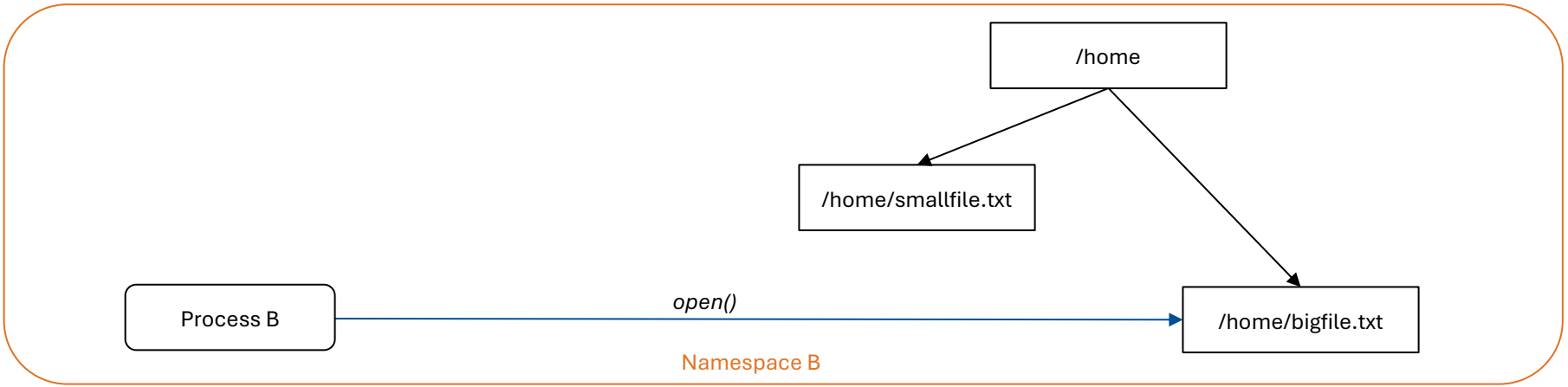
# Containers in General | Linux Namespaces

**Example: Different Namespaces**



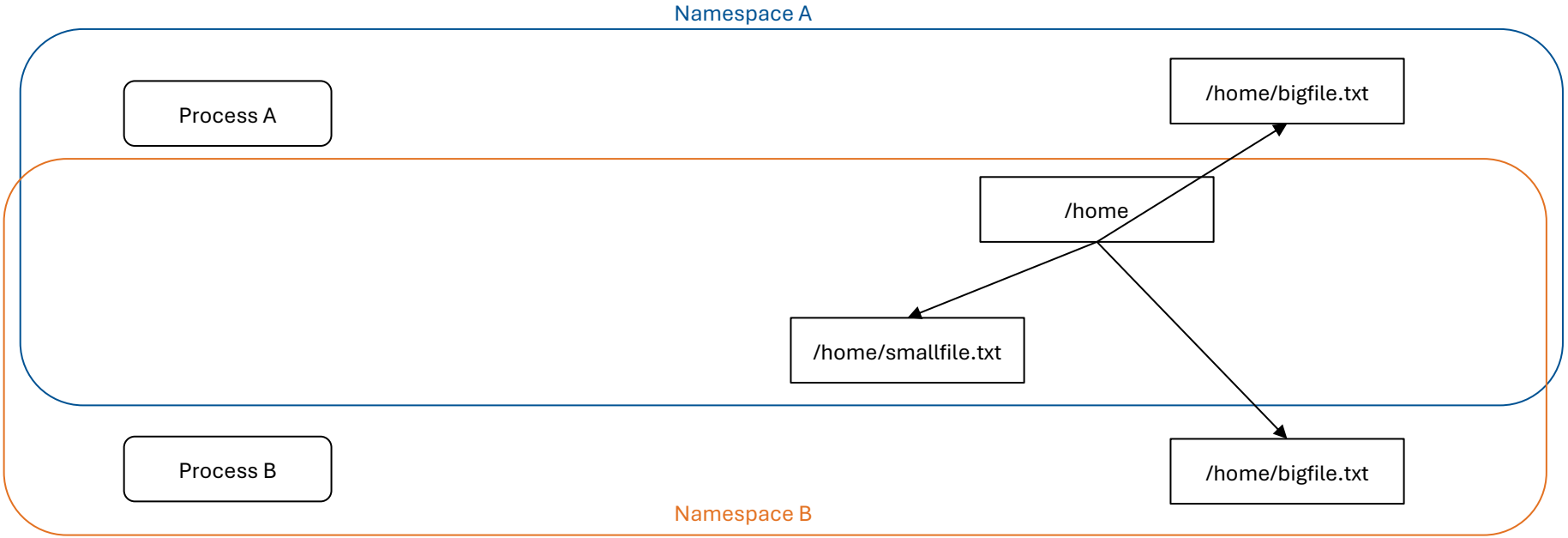
# Containers in General | Linux Namespaces

**Example: Different Namespaces**



# Containers in General | Linux Namespaces

## Example: Different Namespaces



# Containers in General | Linux Namespaces

| Namespace | Resource |
|---|---|
| Cgroup | Cgroup root directory |
| IPC | Message queues, semaphores, and shared memory |
| Network | Network devices, stacks, ports, etc. |
| Mount | Mount points → Filesystem |
| PID | Process Ids |
| Time | Boot and monotonic clocks |
| User | User and group IDs |
| UTS | Hostname and NIS domain name |

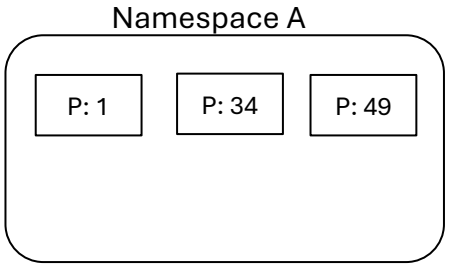
# Containers in General | Linux Namespaces

**Not all namespaces are built equal!**

- (Mainly) **User** and **PID** namespaces are **hierarchical**
- Hierarchical Namespaces are nested:
  - Newly created namespace is child of namespace where it was created
  - Tree structure
  - Parent namespace retains capabilities over child namespace

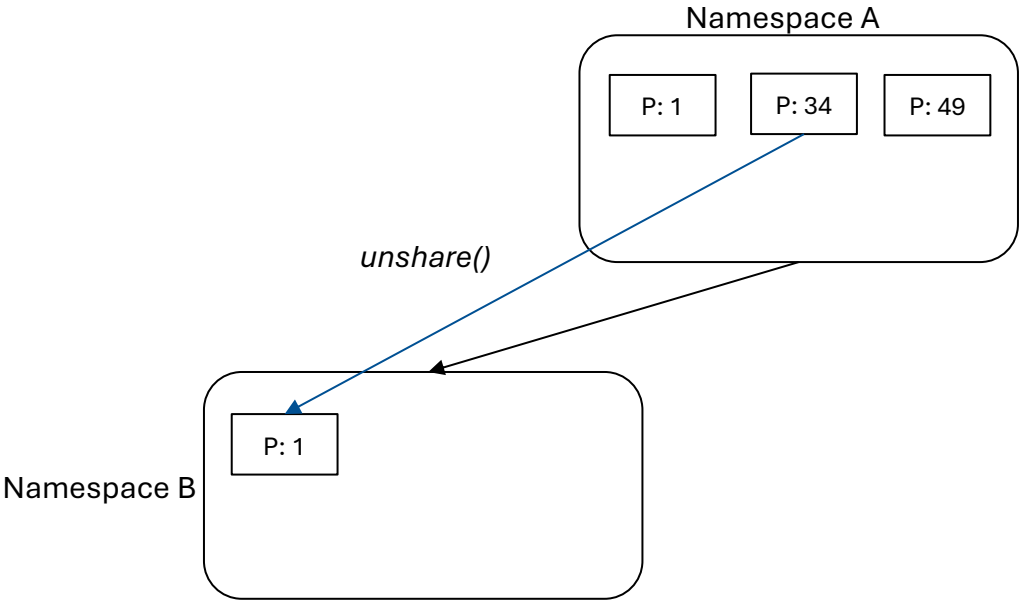
# Containers in General | Linux Namespaces

**Example:**



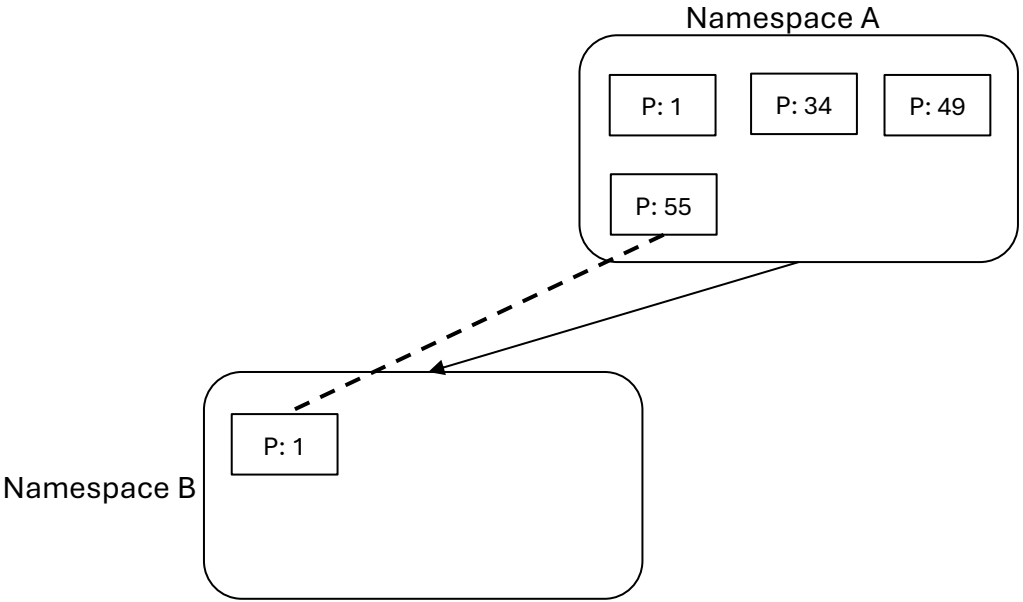
# Containers in General | Linux Namespaces

**Example:**



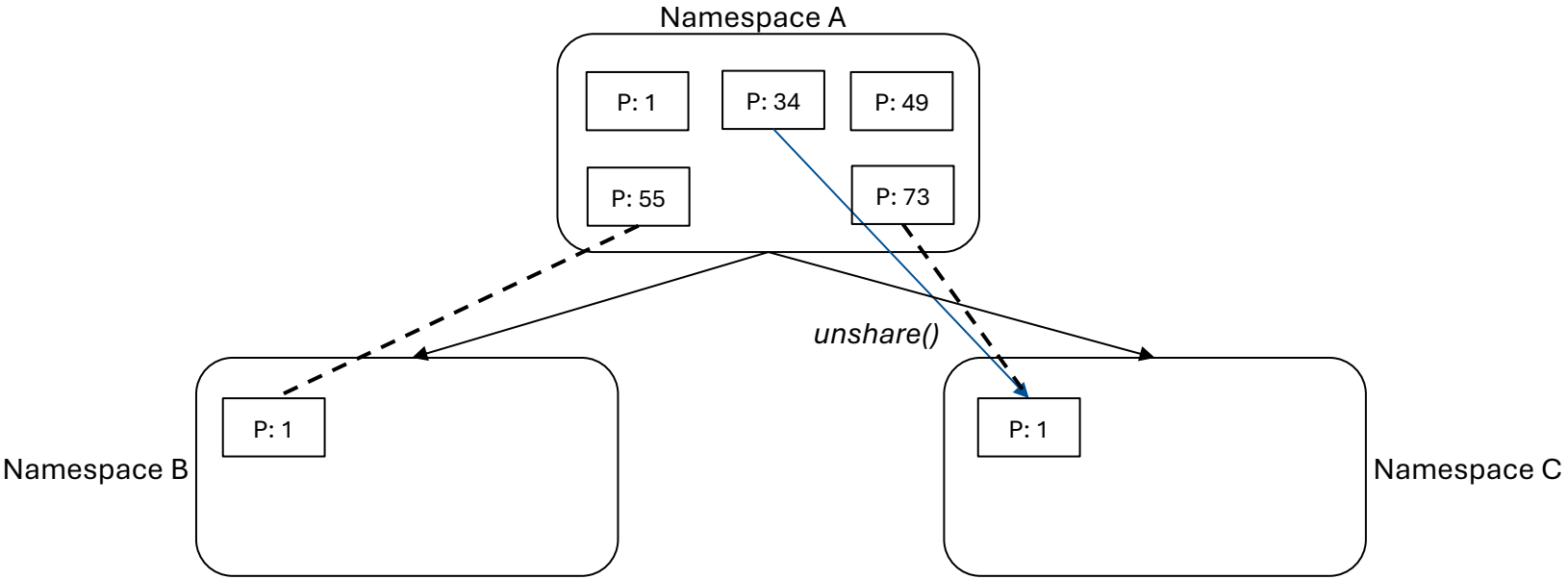
# Containers in General | Linux Namespaces

**Example:**



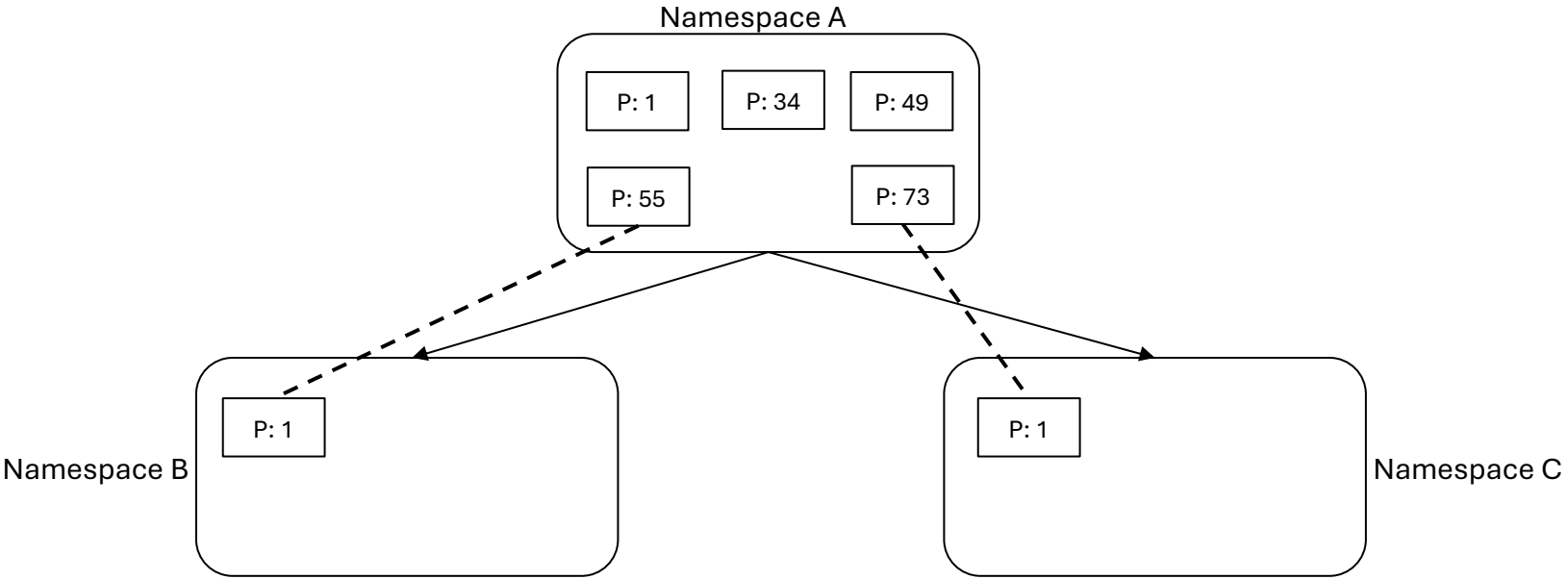
# Containers in General | Linux Namespaces

**Example:**



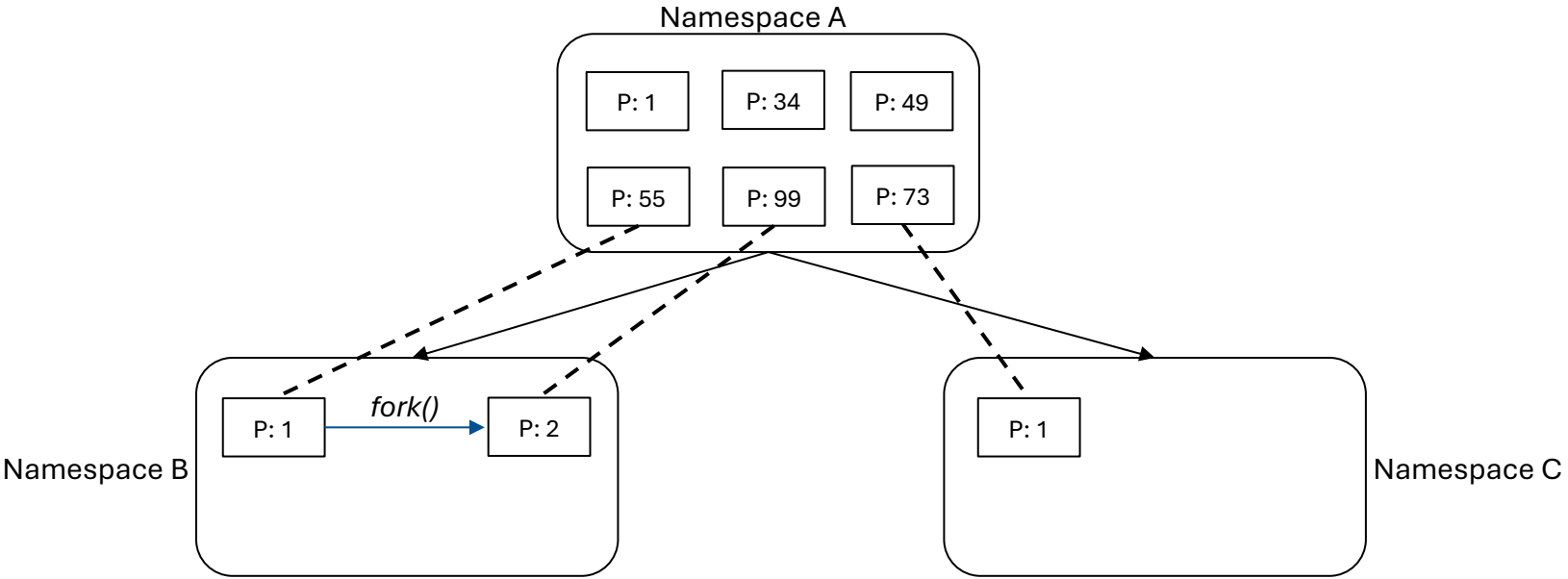
# Containers in General | Linux Namespaces

**Example:**



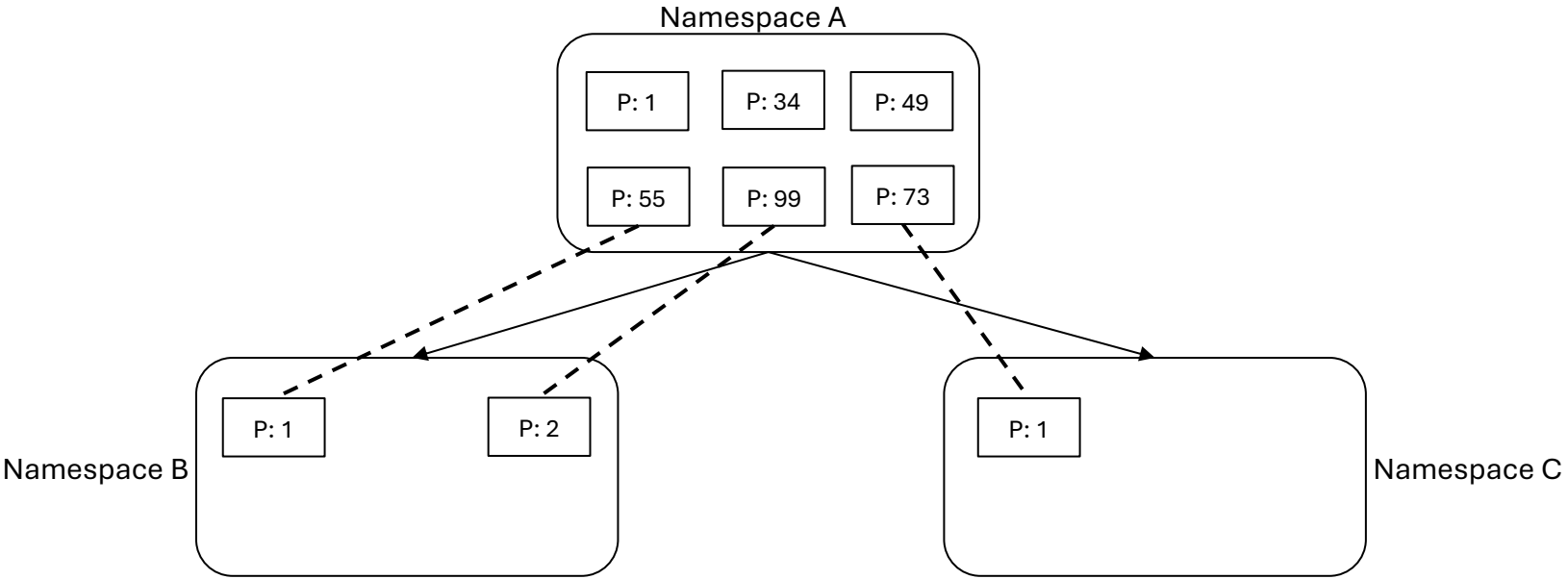
# Containers in General | Linux Namespaces

**Example:**



# Containers in General | Linux Namespaces

**Example:**



# Containers in General | Linux Namespaces

Namespaces: “Hide” Processes from each other → Still compete for resources

How can we solve this?

## Containers in General | Linux Namespaces

Namespaces: “Hide” Processes from each other → Still compete for resources

How can we solve this?

# Linux Control Groups

# Containers in General | Linux Control Groups

**Cgroups (Linux Control Groups):**

- Linux kernel feature
- Order processes into (hierarchical) groups → **limit and monitor resource usage (per group)**
- Interface provided through pseudo-filesystem
- Two versions: cgroups v1 & cgroups v2:
  - V2 created because of issues with V1 → still exists and won't be removed (most likely)

# Containers in General | Linux Control Groups

**Cgroups (Linux Control Groups):**

- (Some) Important resources that can be limited:

| Resource | Details |
|---|---|
| CPU | Upper and lower limits to CPU time |
| Memory | Reporting and limiting of process memory, kernel memory, and swap |
| I/O | Controls and limits access to devices → throttling & upper limits |
| Network Priority | Priorities, per network interface (only in V1 → moved to iptables for V2) |
| Processes | Number of processes that can be created |

# Containers in General | Containers

**Summary:**

- With Container Images, Linux Namespaces, and Cgroups, we can:
  - Isolate a process from the rest of the system (Linux Namespaces)
  - Limit its access to system resources (Cgroups)
  - Replicate a pre-built environment and application (Container Images)

Application running in a specific isolated Environment! → **Container**

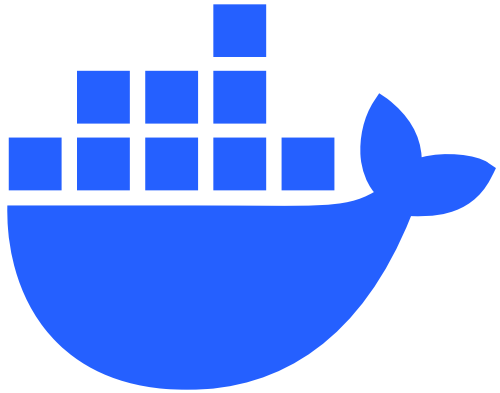
# Containers in General | Container vs VM

**Isolated Environments:**

- Virtual machines and Containers both isolate systems → What is the difference?

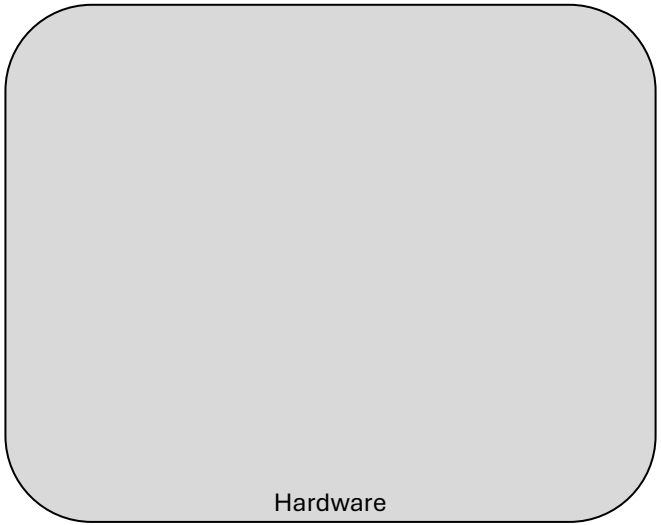

Source: https://commons.wikimedia.org/wiki/File:Kvmbanner-logo2_1.png

VS



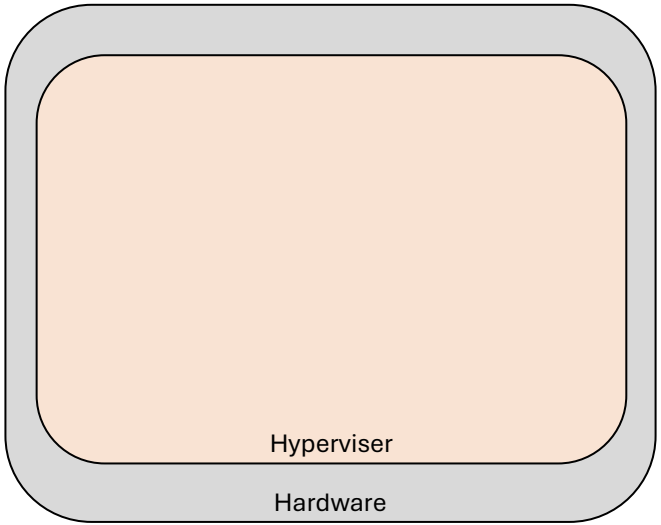
Source: https://www.docker.com/company/newsroom/media-resources/

# Containers in General | Container vs VM



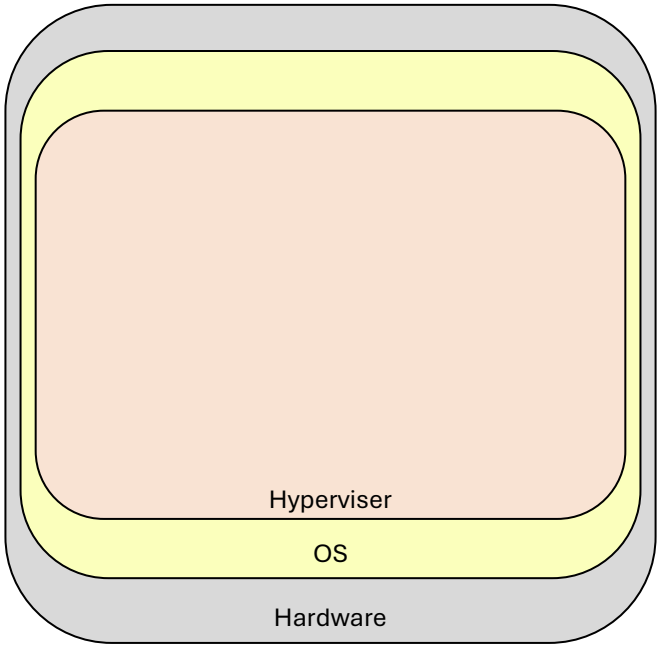

(VM) Virtualization

# Containers in General | Container vs VM



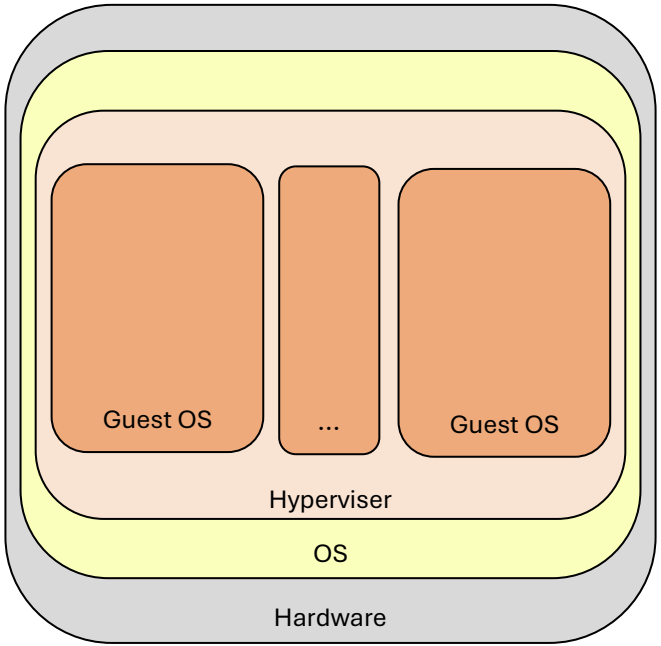

(VM) Virtualization

# Containers in General | Container vs VM



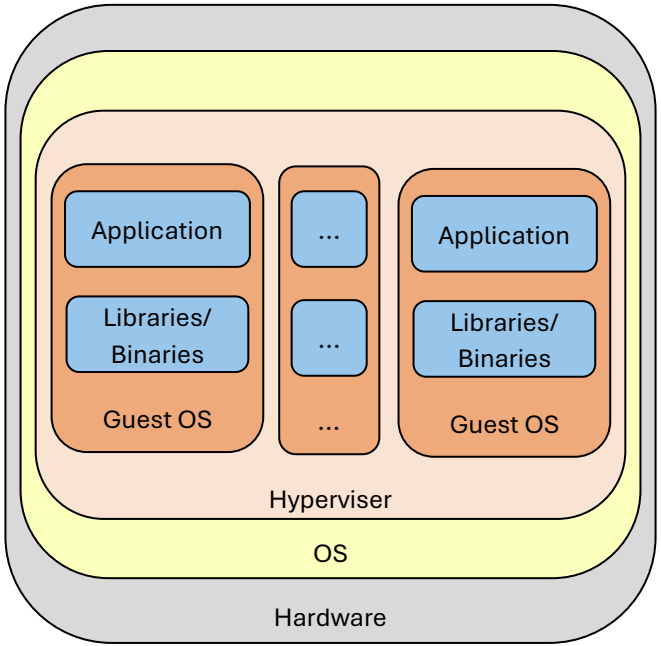

(VM) Virtualization

# Containers in General | Container vs VM



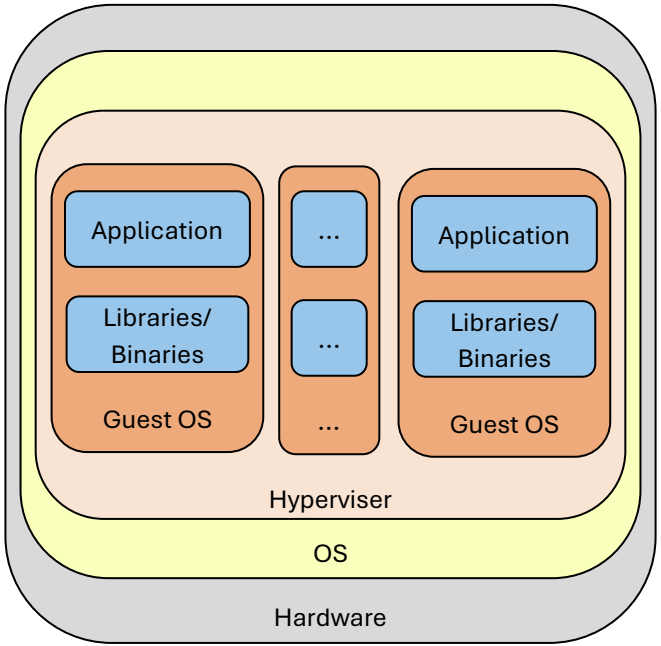

(VM) Virtualization

# Containers in General | Container vs VM

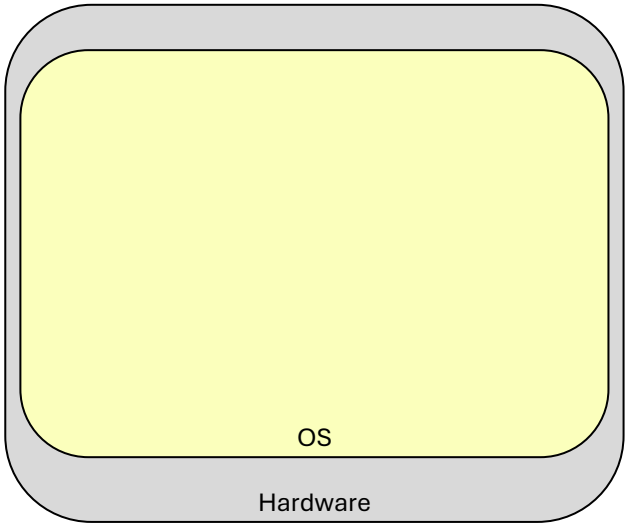



(VM) Virtualization

# Containers in General | Container vs VM

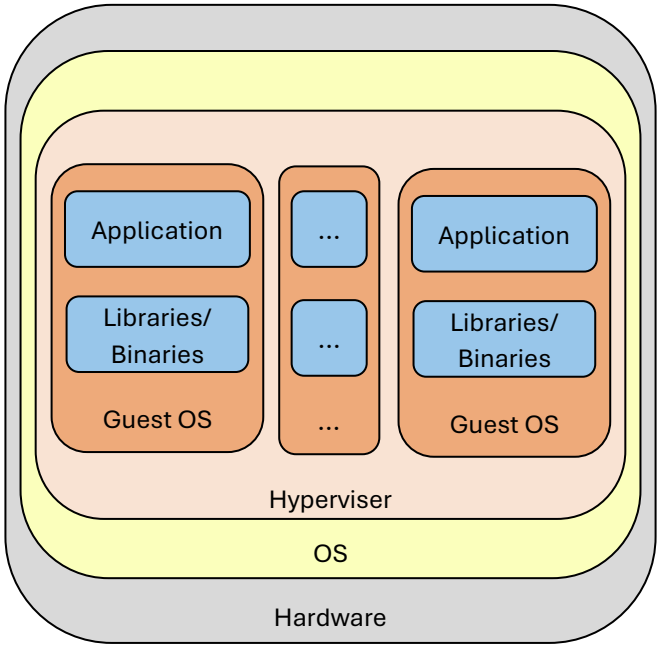



(VM) Virtualization

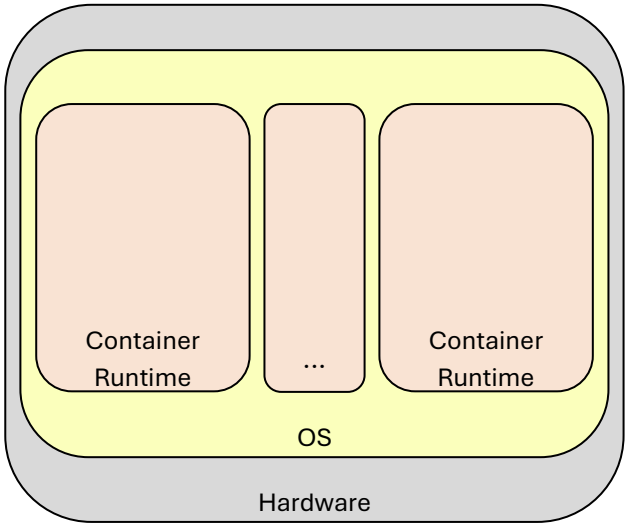



Containerization

# Containers in General | Container vs VM

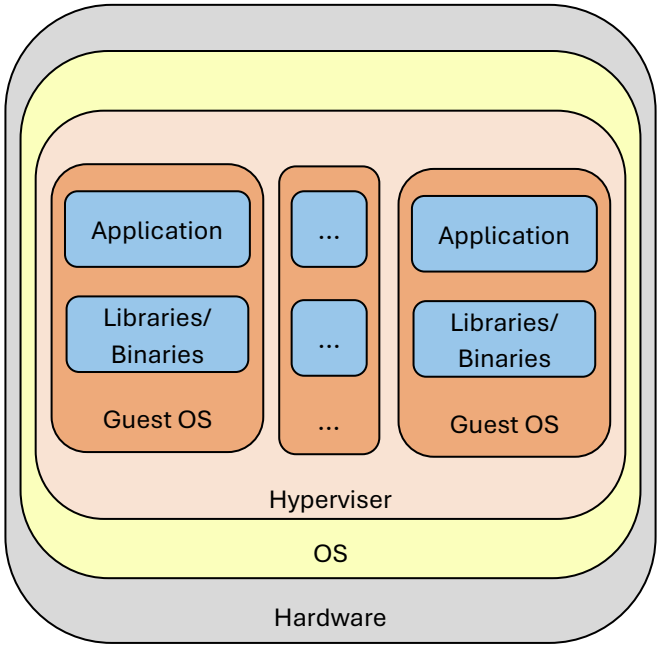



(VM) Virtualization

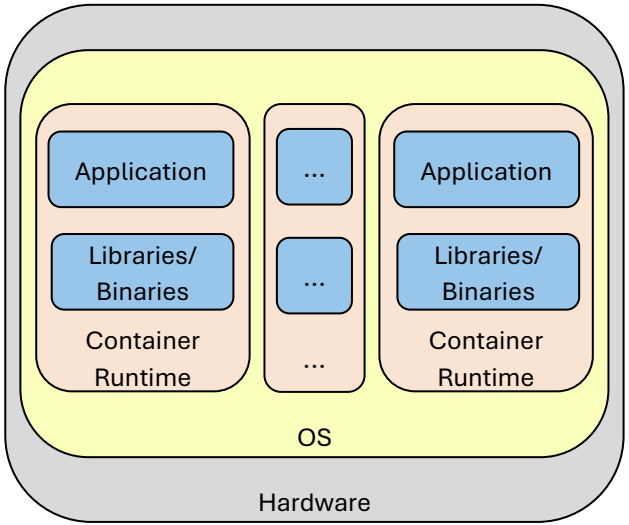



Containerization

# Containers in General | Container vs VM

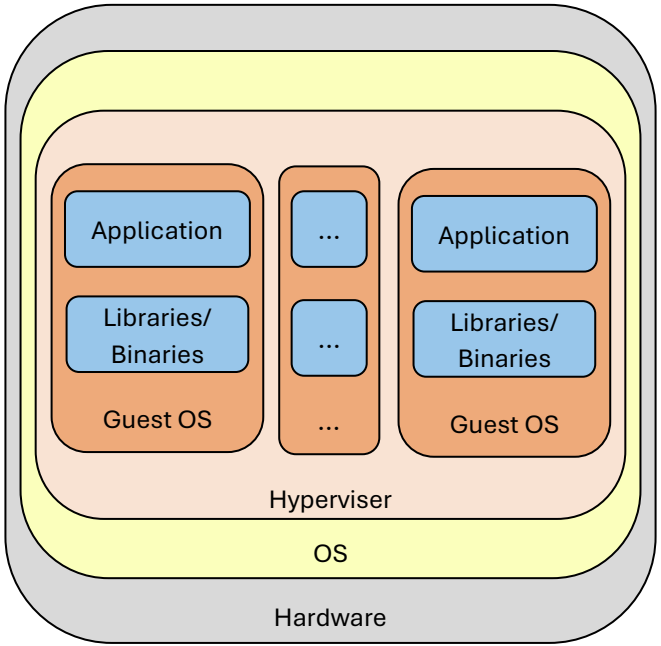



(VM) Virtualization

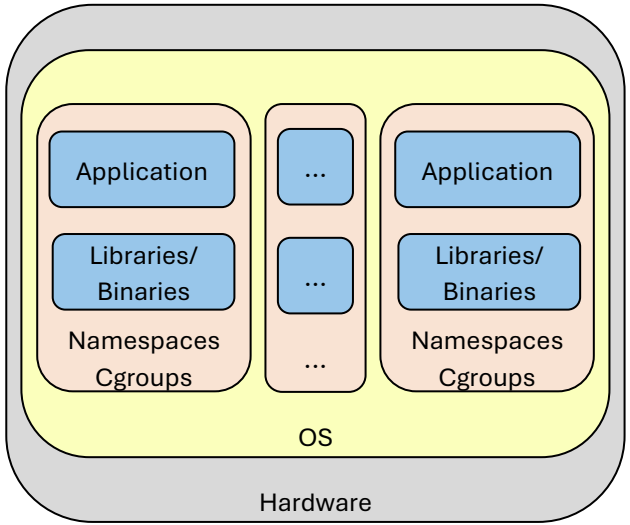



Containerization

# Containers in General | Container vs VM




(VM) Virtualization




Containerization

# Containers in General | Container vs VM

Main Differences:

| | Virtual Machines | Containers |
|---|---|---|
| Resource Access | Hypervisor | Cgroups |
| System Isolation | Seperate OS | Linux Namespaces |
| Kernel | Seperate | Shared |
| Installation | Provision VM + Full OS install | Call to unshare() + cgroup entry |
| Size | OS + Libraries/Binaries + Application Size | Libraries/Binaries + Application Size |

# Containers in General | Container vs VM

Advantages and Disadvantages:

| | Virtual Machines | Containers |
|---|---|---|
| Setup | Time-consuming (VM + Full OS install) | Quick (Systemcalls + cgroups) |
| Migration | Time-consuming, hyperviser dependant | Quick |
| Isolation | Fully isolated (sometimes hardware based) | Limited isolation: shared kernel, etc. |
| Updates | Regular OS and software updates possible | Often requires new image |
| Size | OS + Libraries/Binaries + Application Size | Libraries/Binaries + Application Size |

# Containers in General | Container vs VM

Summary:

- Containers are more lightweight, i.e., faster startup, fewer resources required, easier to migrate
- VMs provide full isolation → more secure, more stable, not impacted (as much) by other VMs/Containers

Usage:

- Containers: microservices, stateless applications, short lifetimes (e.g., frequent updates)
- VMs:  legacy applications, high stability, Kernel-Dependent, specially optimized

# Container Environment

# Introduction

Now we know what a container is and how they can be implemented!

Manual container setup:

- Required OS and Kernel-Level domain knowledge → only for specialists
- Error-prone, easy to misconfigure, and miss something → security and functionality issues
- Time-consuming → reduces flexibility and efficiency of containers

Solution → automation through **auxiliary tools and environments!**

# Introduction

Typical container workflow:

1. **Get** and or **build** an image
2. **Run** the container
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container

# Introduction

Typical container workflow:

1. **Get** and or **build** an image → Container registries and build tools (e.g., Dockerfiles)
2. **Run** the container → Container runtimes
3. **Maintain** and **monitor** the container → Auxiliary tools e.g., by Docker or Podman
4. **Destroy** or replace the container → Container runtimes

# Container Registries

Where do we get our images?

# Container Registries

Where do we get our images?

- Manual build?

# Container Registries

Where do we get our images?

- Manual build?
- Many applications are based on **standard/preexisting** functionality → already built
- Reuse imagas build by other people

→ Container registries

# Container Registries

What is a container registry?

- Centralized hub for **storing** and **distributing** container images
- Common examples: Docker Hub, Amazon Elastic Container Registry, Azure Container Registry
- Registries can be public or private (e.g., company internal registries)
- Specification: OCI Distribution Specification

# Container Registries

Important parts of the specification:

MUST

- **Pull:** download artifacts (i.e., images) from the registry

SHOULD

- **Push:** upload artifacts (i.e., images) to the registry
- **Content Discovery:** list or otherwise query content stored in the registry
- **Content Management:** control the full life-cycle of the content stored in the registry

# Container Registries

Demo:

**python** Docker Official Image · 1B+ · 10K+

Python is an interpreted, interactive, object-oriented, open-source programming language.

LANGUAGES & FRAMEWORKS

Source: https://hub.docker.com/_/python/

# Introduction

Typical container workflow:

1. **Get** and or **build** an image
2. **Run** the container
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container

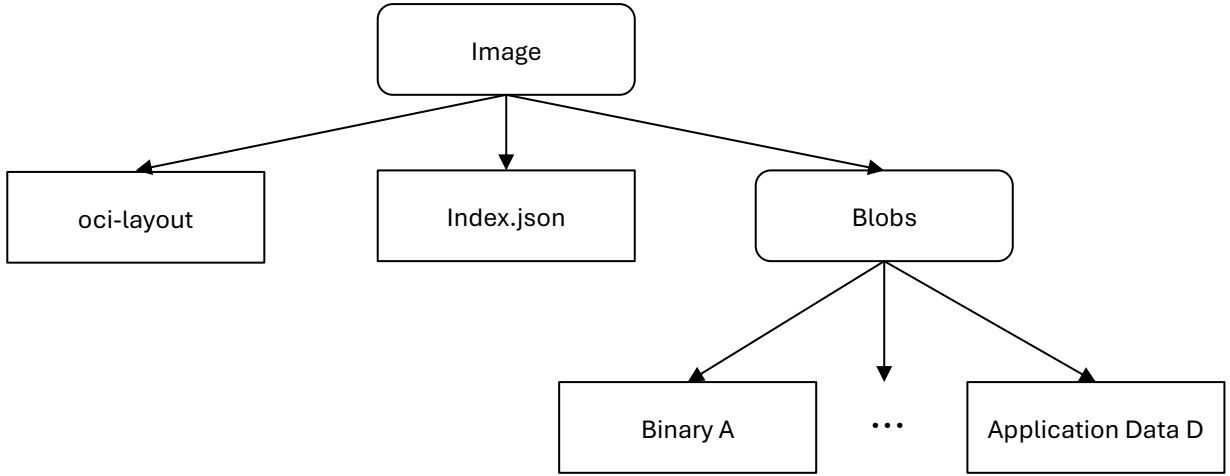
# Introduction

Typical container workflow:

1. **Get** and or **build** an image
2. **Run** the container
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container

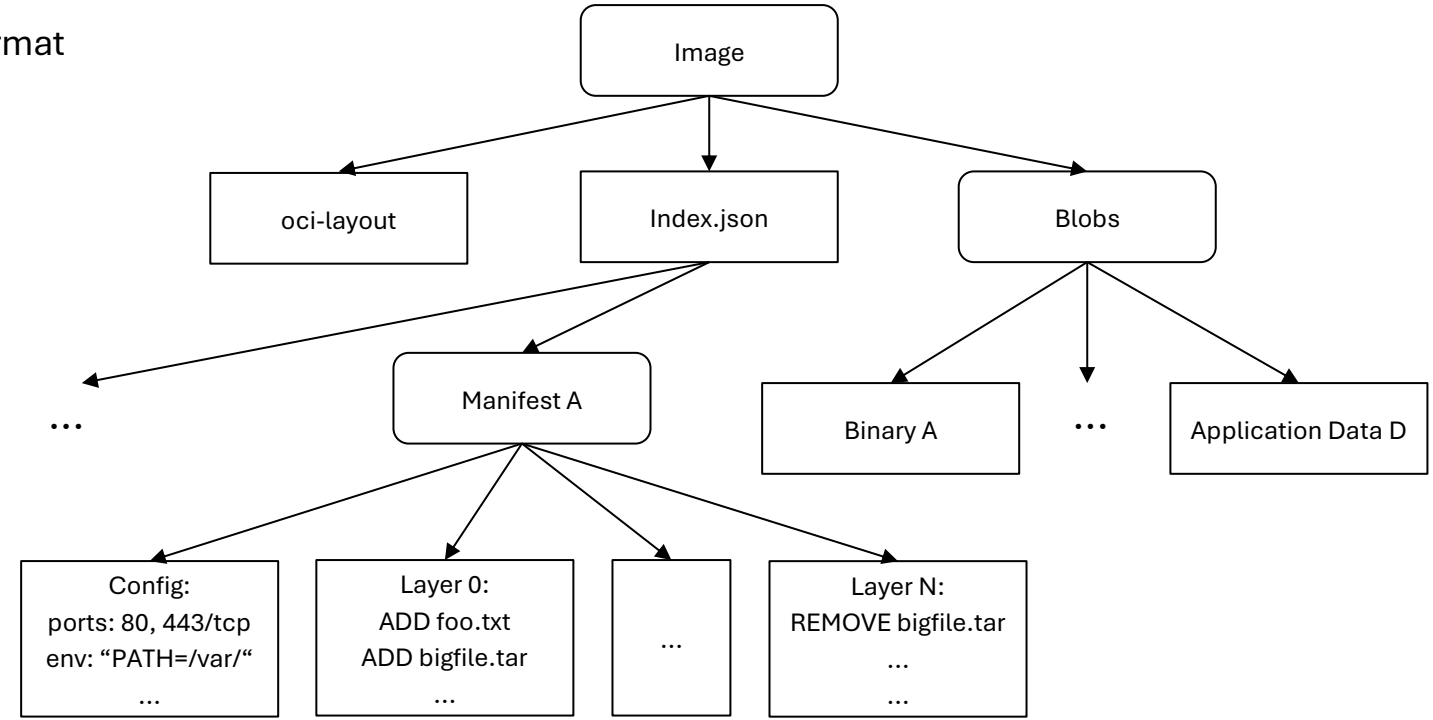
# Building Images

**Reminder:** OCI Image format



# Building Images

**Reminder:** OCI Image format



# Building Images

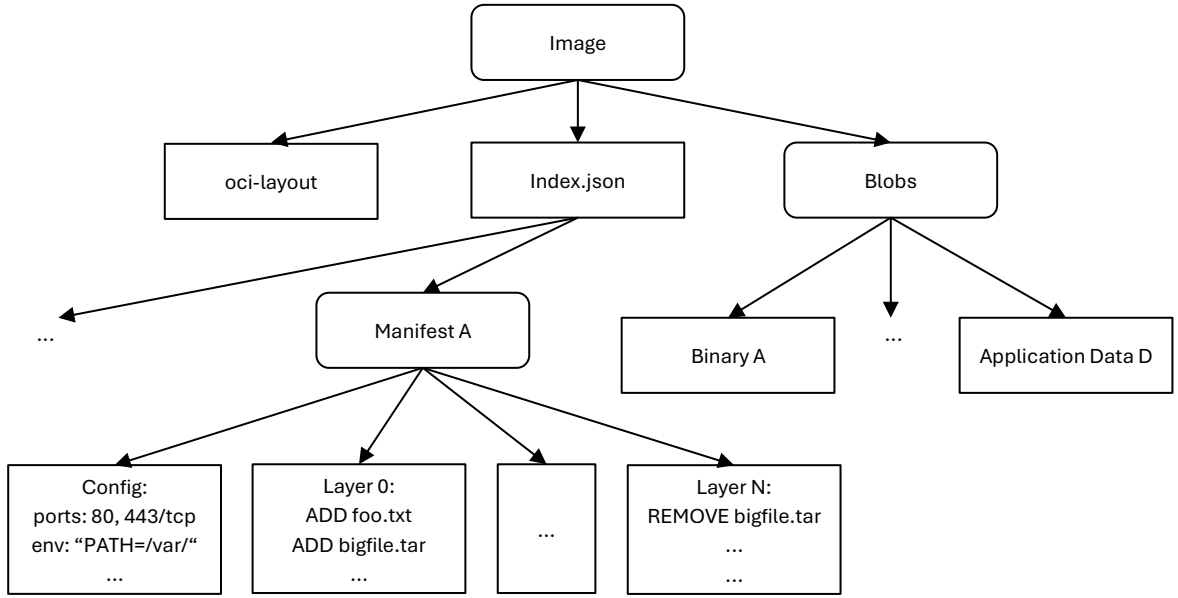
**Reminder:** OCI Image format

Manual build unrealistic:

- Time-consuming, error-prone
- Requires expert system knowledge

Solution:

- Automation



# Building Images

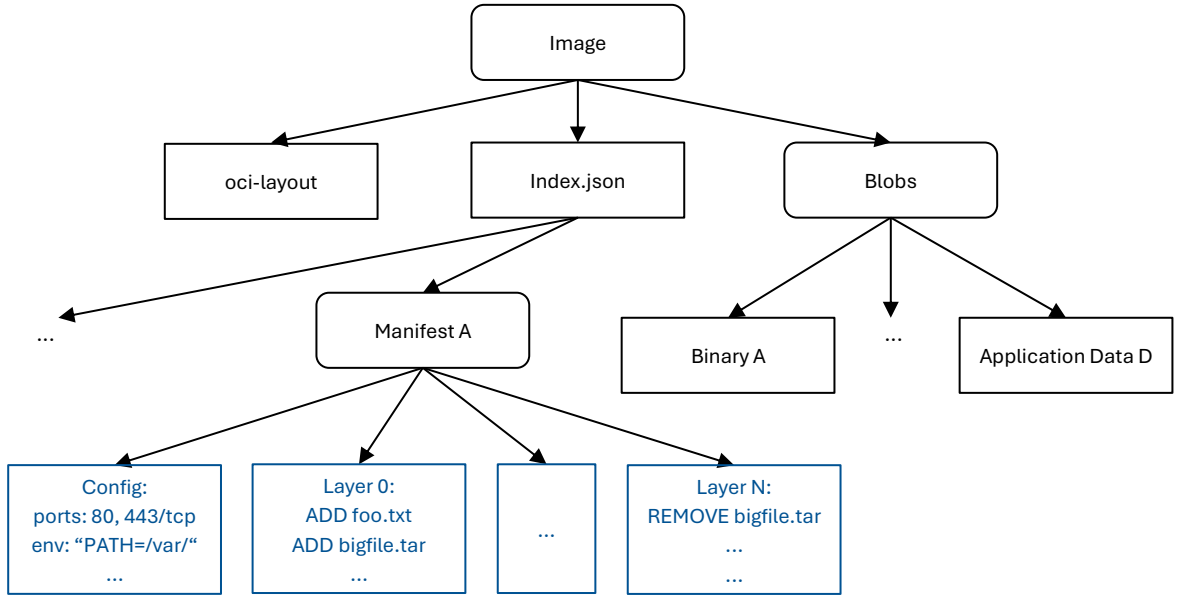
**Reminder:** OCI Image format

Manual build unrealistic:

- Time-consuming, error-prone
- Requires expert system knowledge

Solution:

- Automation
- Re-use preexisting images → layered structure



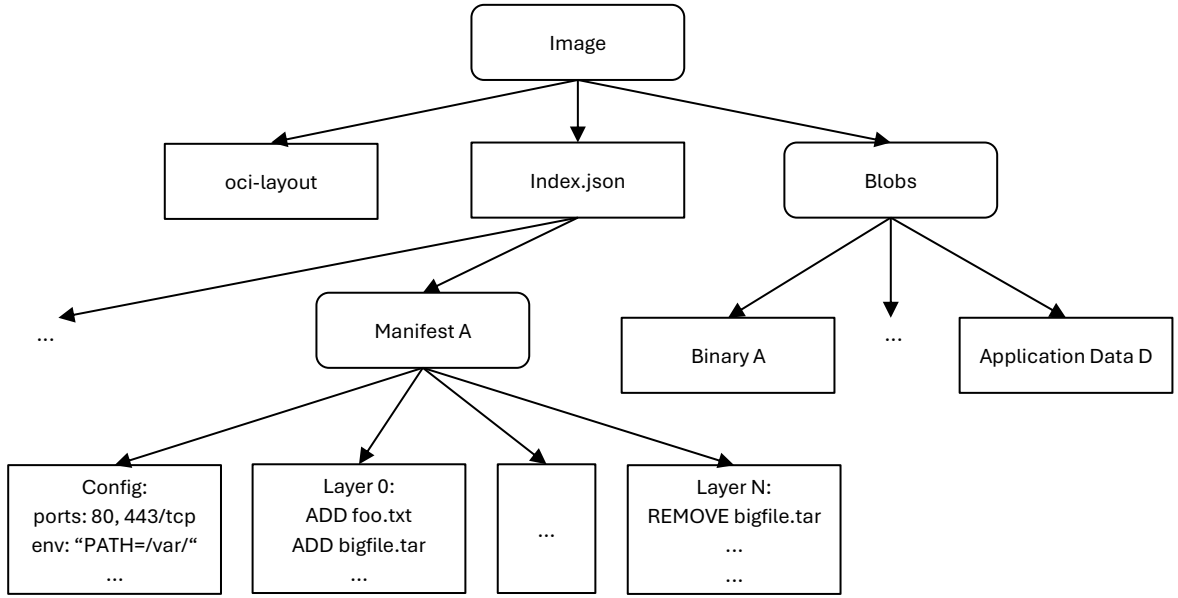
# Building Images

Building images (simplified):

# Building Images

Building images (simplified):

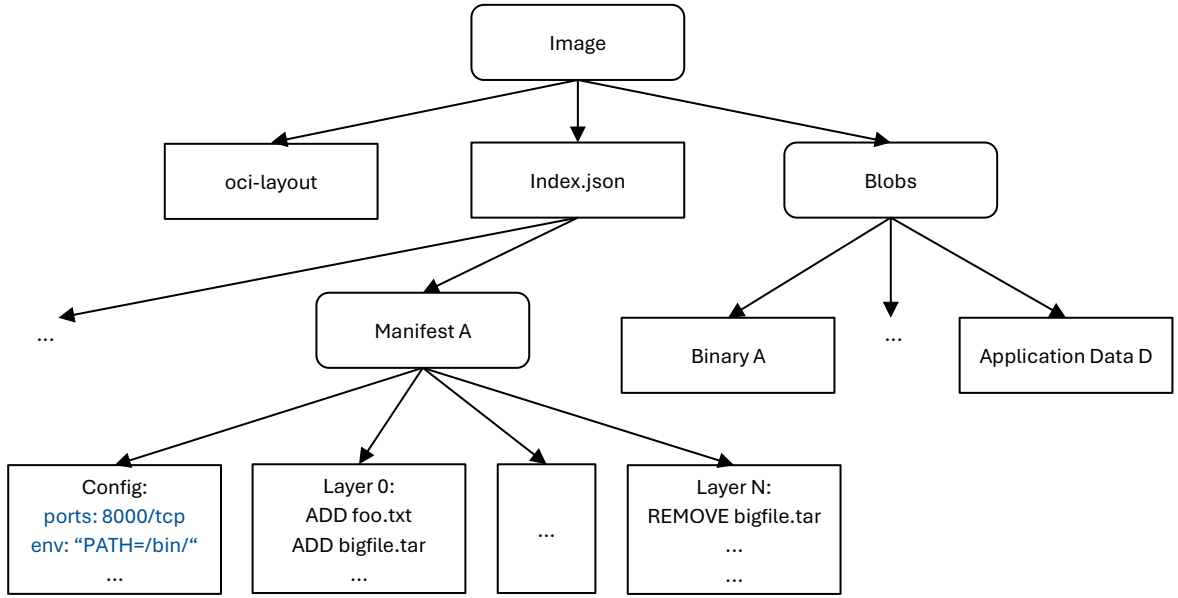
1. Pull preexisting image from registry



# Building Images

Building images (simplified):

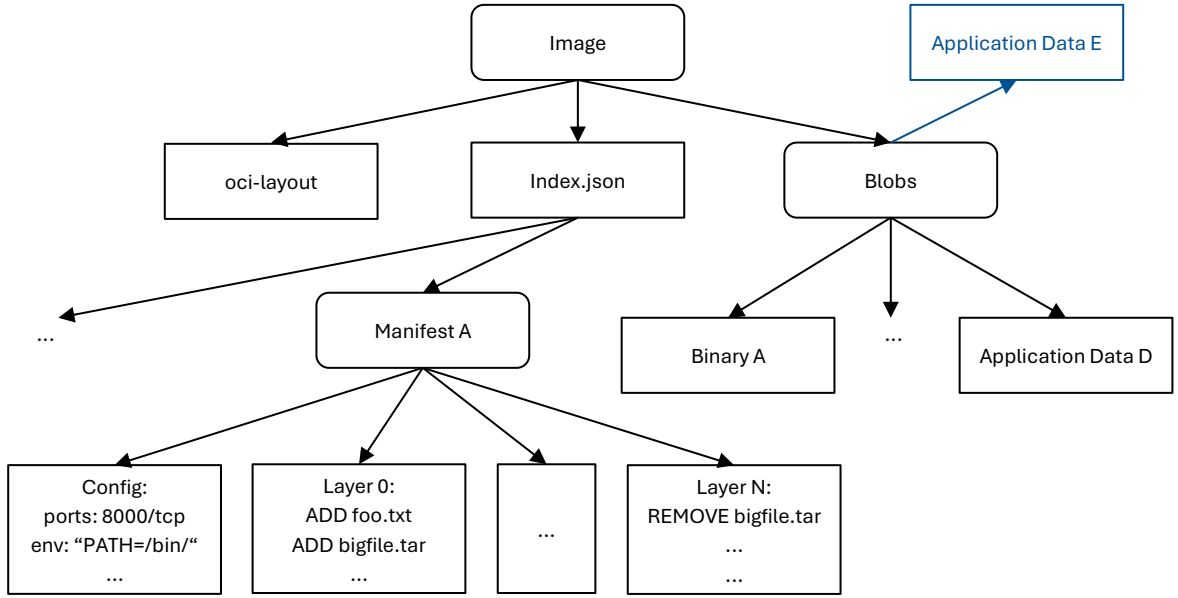
1. Pull preexisting image from registry
2. Adjust Config & Metadata



# Building Images

Building images (simplified):

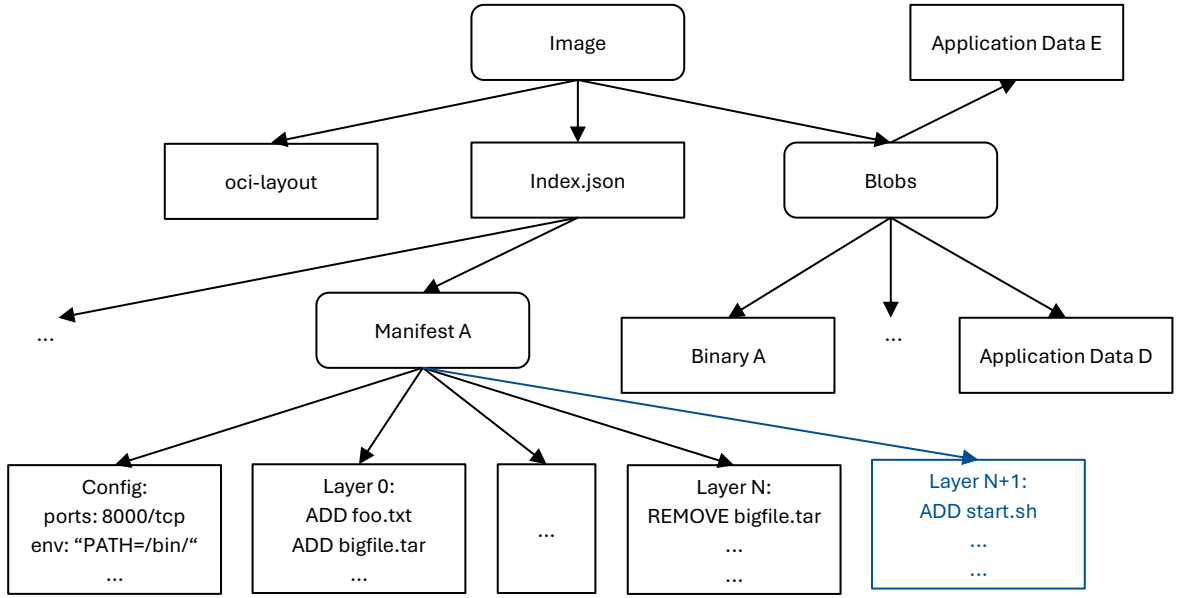
1. Pull preexisting image from registry
2. Adjust Config & Metadata
3. Add necessary blobs



# Building Images

Building images (simplified):

1. Pull preexisting image from registry
2. Adjust Config & Metadata
3. Add necessary blobs
4. Add new Layer



# Building Images

Showcase: **Dockerfiles**

- File format supported by Docker (and many others)
- Used as „recipe “ to build images automatically
- Human-readable

# Building Images

Showcase: **Dockerfiles**

- Example:

```
FROM python:latest

WORKDIR /usr/src/app

COPY example.py index.html ./

EXPOSE 5000

CMD [ "python", "./example.py" ]
```

# Introduction

Typical container workflow:

1. **Get** and or **build** an image
2. **Run** the container
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container

# Introduction

Typical container workflow:

1. **Get** and or **build** an image 
2. **Run** the container
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container

# Running Containers

**Reminder:** Creating a container

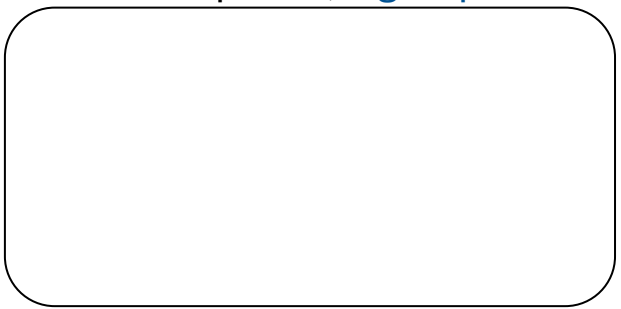
# Running Containers

**Reminder:** Creating a container

# Running Containers

**Reminder:** Creating a container

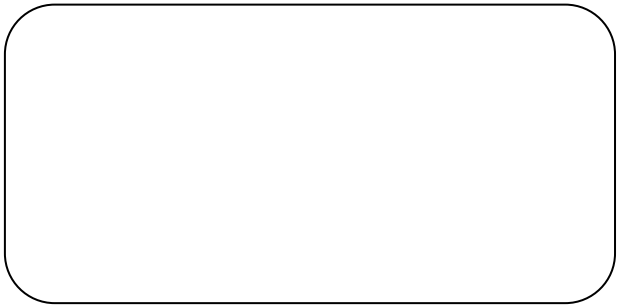
Namespaces, Cgroups

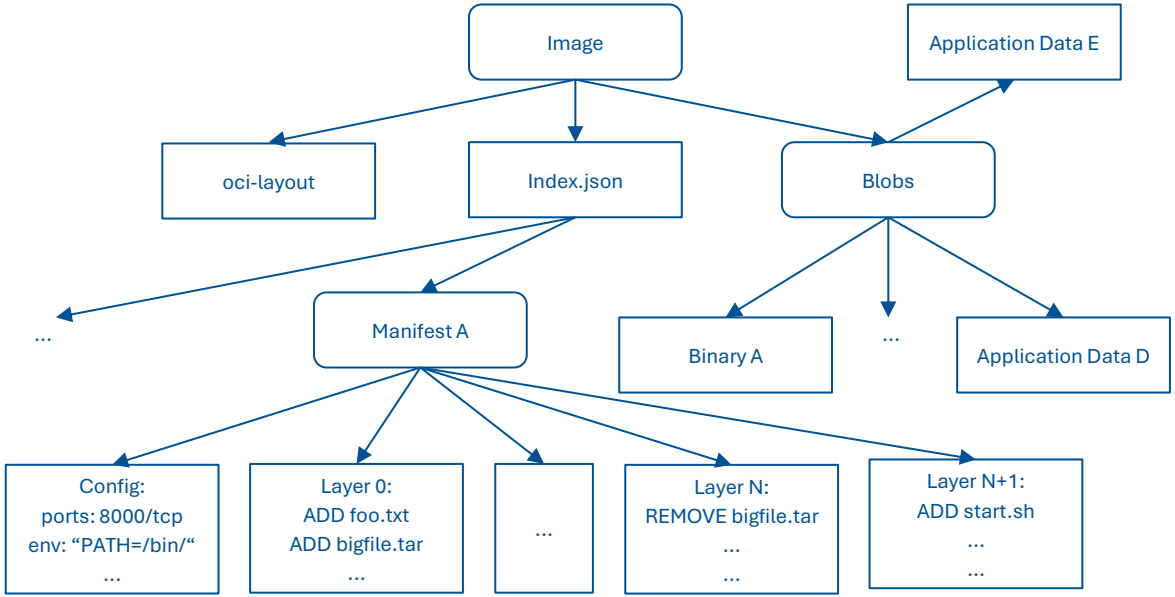


# Running Containers

**Reminder:** Creating a container

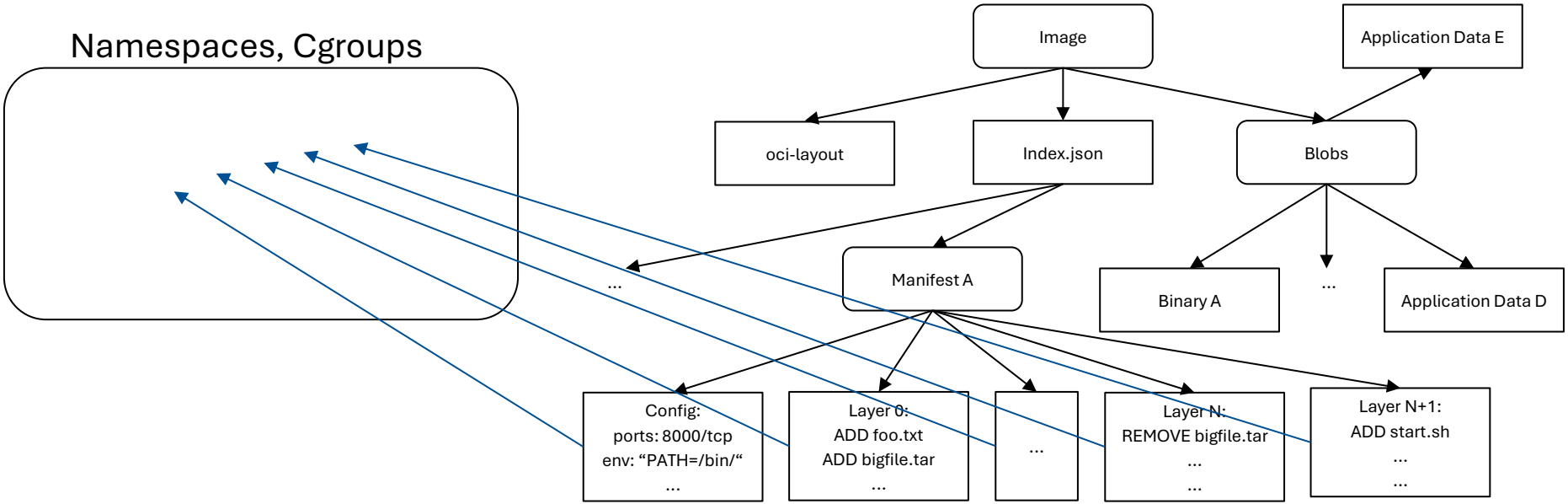
Namespaces, Cgroups





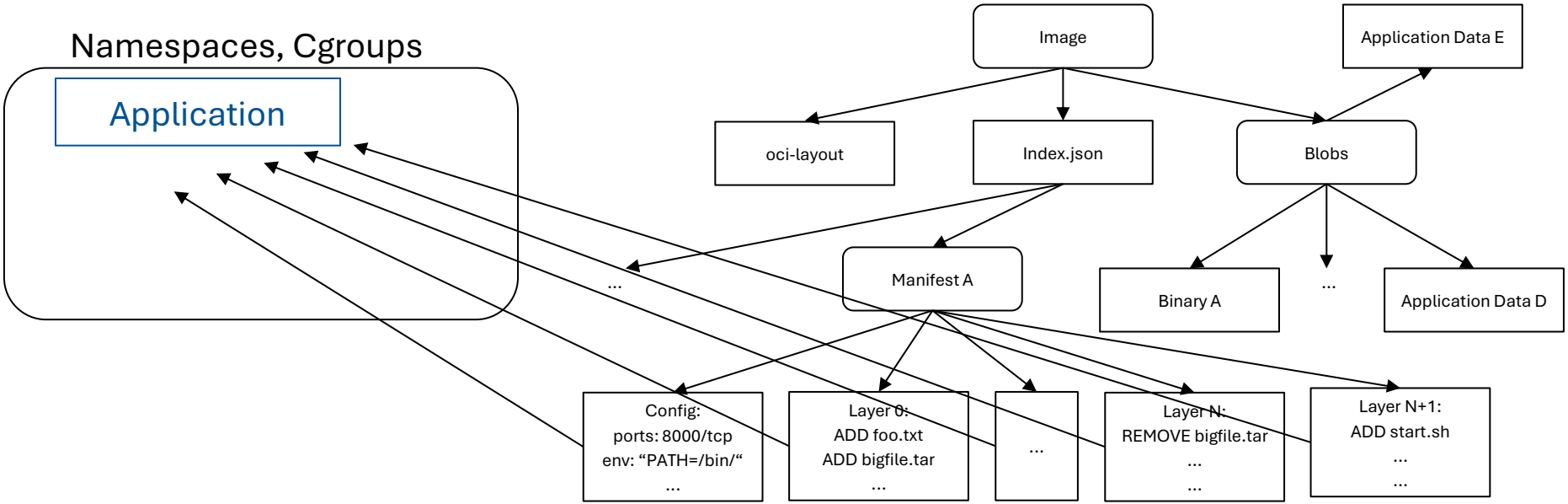
# Running Containers

**Reminder:** Creating a container



# Running Containers

**Reminder:** Creating a container



# Running Containers

## Container Runtimes

# Running Containers

**Container Runtimes** → OCI Specification:

Defines:

- Configuration
- Execution environment
- Lifecycle of a container

# Running Containers

OCI Specification:

- Encoding format for containers → **filesystem bundle**
- All necessary data (and metadata) to **load and run** the container:
  - The container's root filesystem
  - MUST include a file named config.json
  - …

# Running Containers

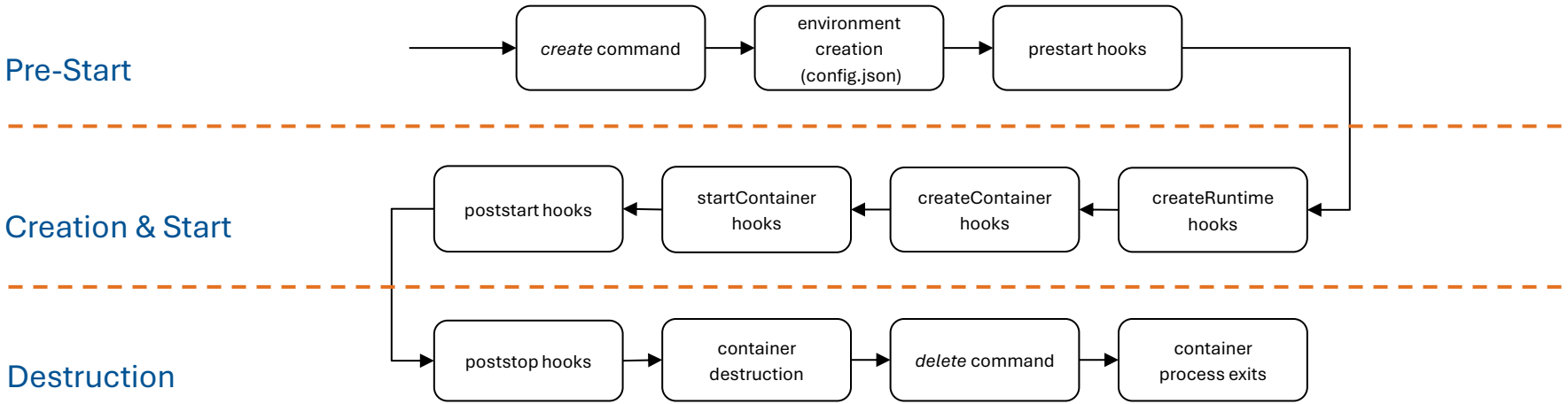
OCI Specification:

- config.json → defines runtime **environment** of the container:
  - Path to containers' root filesystem
  - Other mounts
  - Container process
  - Hooks (actions related to container lifecycle, e.g., createContainer, startContainer)
  - …

# Running Containers

OCI Specification:

- Container **lifecycle**:



# Running Containers

Implementations:

- Docker uses containerd by default → uses runc
- Other OCI-compliant runtimes:
  - crun (lightweight)
  - Runv (hypervisor-based) → obsoleted by Kata Containers
  - Youki (Rust)

# Running Containers

Showcase: **Docker Run**

## docker container run

Copy as Markdown

| | |
|---|---|
| **Description** | Create and run a new container from an image |
| **Usage** | `docker container run [OPTIONS] IMAGE [COMMAND] [ARG...]` |
| **Aliases** | `docker run` |

Source: https://docs.docker.com/reference/cli/docker/container/run/

# Introduction

Typical container workflow:

1. **Get** and or **build** an image 
2. **Run** the container
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container

# Introduction

Typical container workflow:

1. **Get** and or **build** an image ✓
2. **Run** the container ✓
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container ✓

# Docker & Podman

Running containers (even with tools) requires:

- Pulling  & building images
- Running containers
- Monitoring
- …

→ combined tool for **full** container functionality

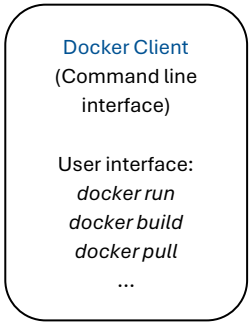
# Docker & Podman

Docker & Podman:

- Provide: pulling & building images, running containers, monitoring, …
- Open source
- Go
- Docker → hosts public image registry: Docker Hub

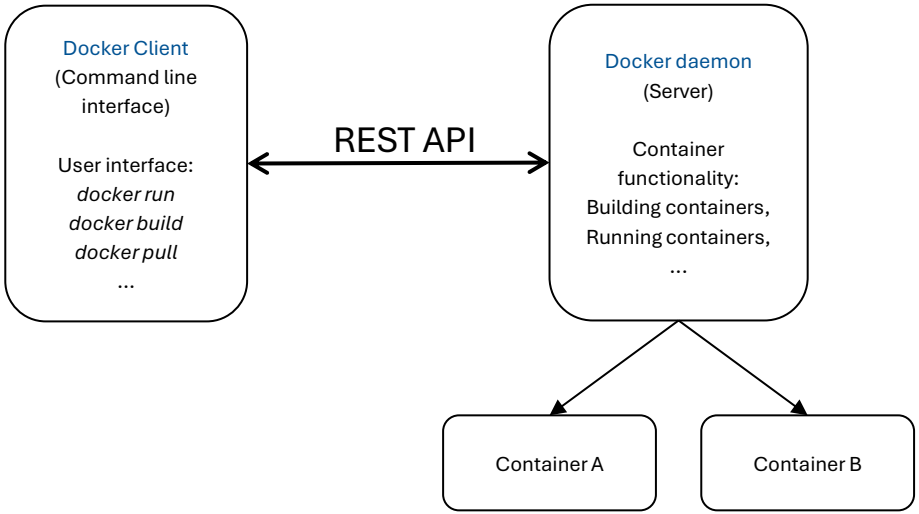
# Docker & Podman

Docker architecture:



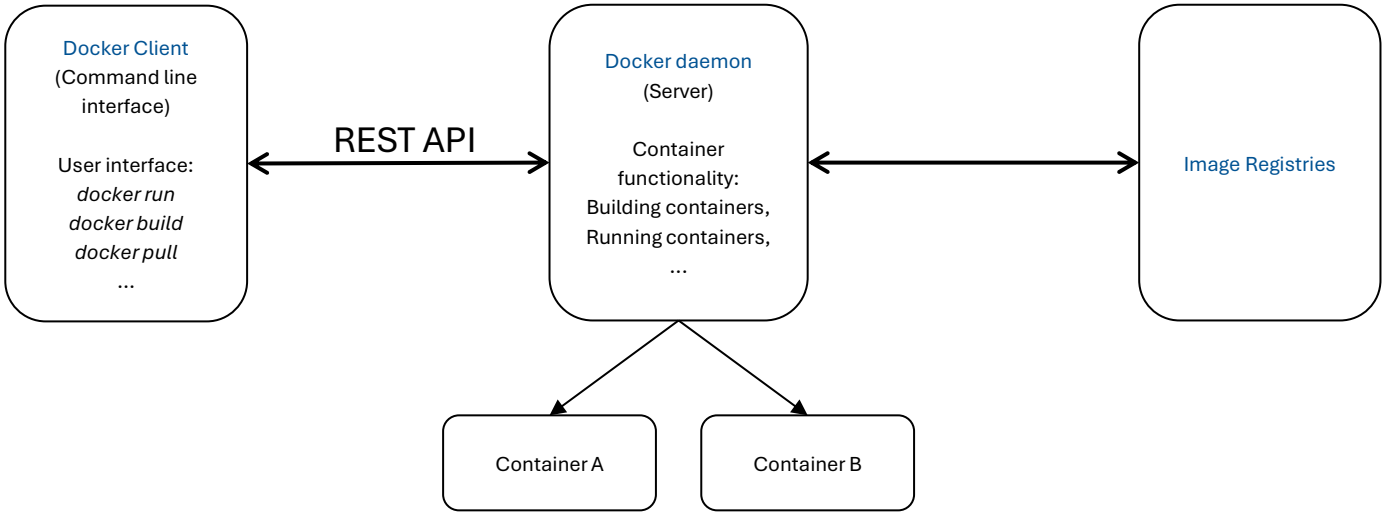
# Docker & Podman

Docker architecture:



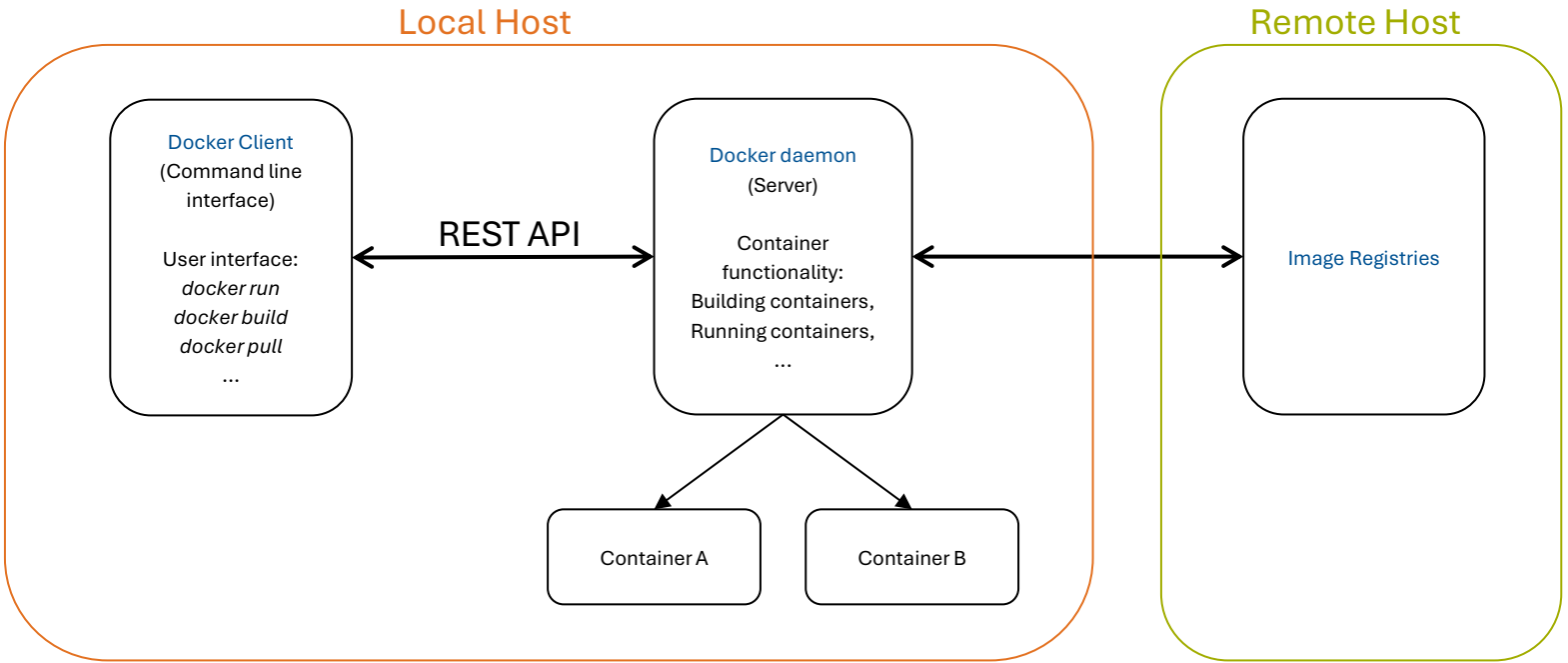
# Docker & Podman

Docker architecture:



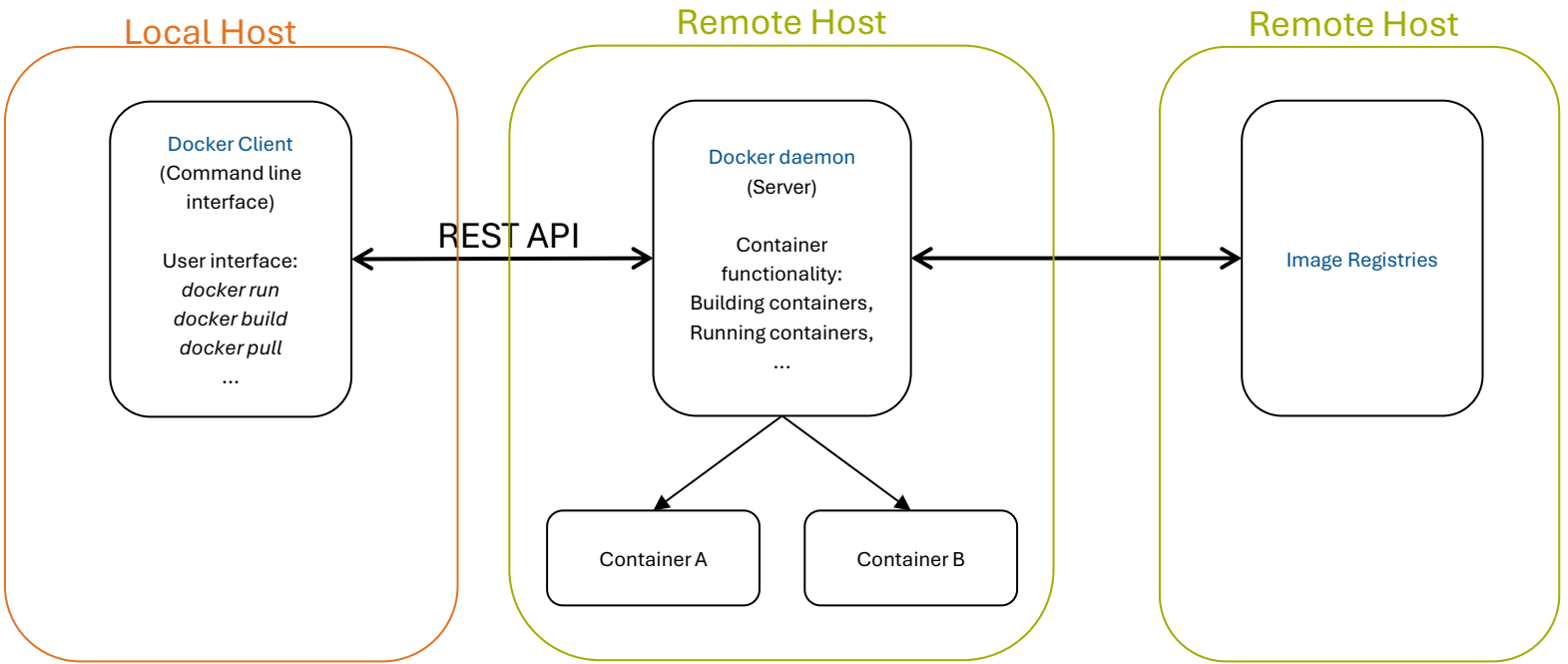
# Docker & Podman

Docker architecture:



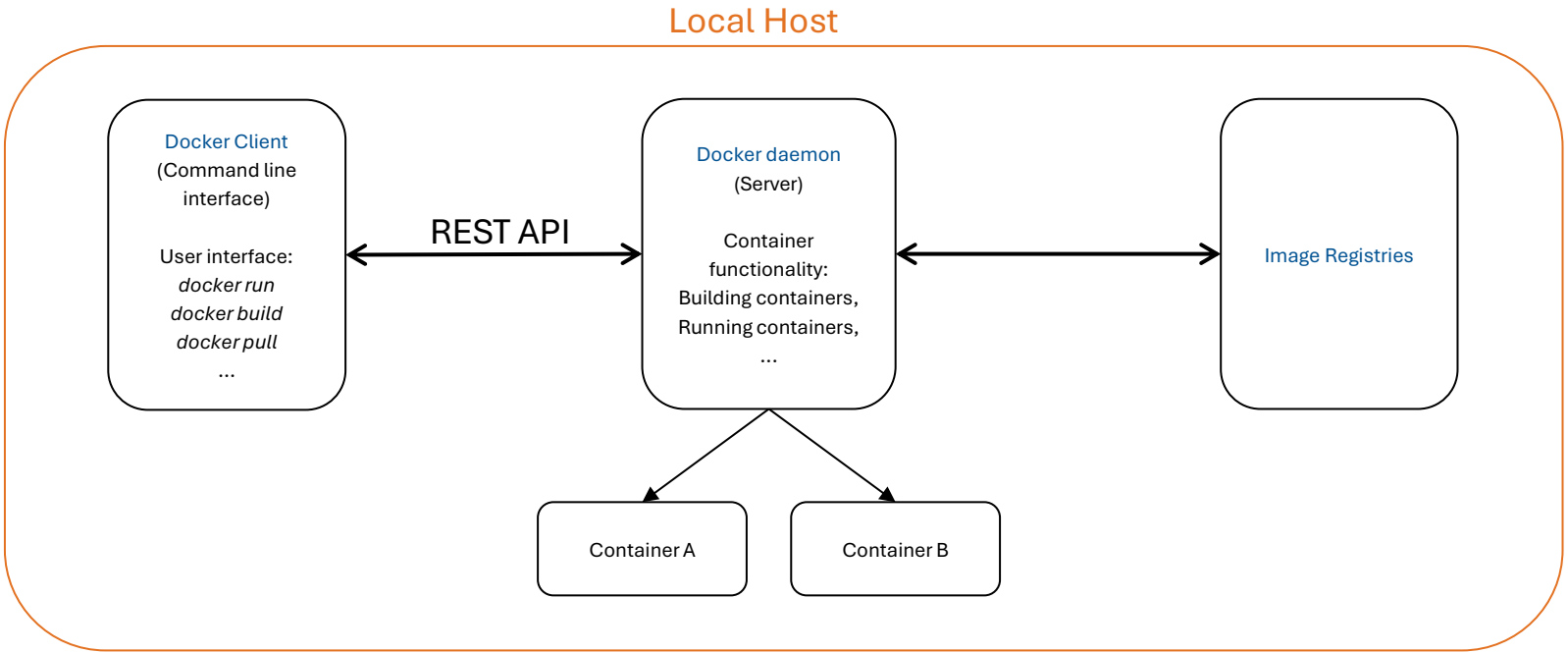
# Docker & Podman

Docker architecture:



# Docker & Podman

Docker architecture:



# Docker & Podman

Docker permission issue:

- Docker daemon (*dockerd*) requires **root privileges!**

# Docker & Podman

Docker permission issue:

- Docker daemon (dockerd) requires **root privileges!** → Container started using root privileges
- Containers share OS Kernel → potential security issue
- Solution → Rootless Mode:
  - Executes Docker daemon and containers in user namespace
  - No root privileges required

# Docker & Podman

Podman:

- Daemonless
- Containers can be executed with or without root privileges (in a user namespace)
- Container **never** has more privileges than the calling user
- Optionally provides podman-remote:
  - Client-server architecture → client connects to server via SSH and uses REST API

# Running Containers

Showcase: **Docker Exec**

## docker container exec

Copy as Markdown

| | |
|---|---|
| **Description** | Execute a command in a running container |
| **Usage** | `docker container exec [OPTIONS] CONTAINER COMMAND [ARG...]` |
| **Aliases** | `docker exec` |

Source: https://docs.docker.com/reference/cli/docker/container/exec/

# Introduction

Typical container workflow:

1. **Get** and or **build** an image ✓
2. **Run** the container ✓
3. **Maintain** and **monitor** the container
4. **Destroy** or replace the container ✓

# Introduction

Typical container workflow:

1. **Get** and or **build** an image ✓
2. **Run** the container ✓
3. **Maintain** and **monitor** the container ✓
4. **Destroy** or replace the container ✓

# Summary

Auxiliary Container Tools:

- Container images can be uploaded and downloaded from container registries
- Images can be built by reusing existing images, e.g., using Dockerfiles
- Containers are run using container runtime environments
- Tools like Docker & Podman combine and add additional functionality

# Sources


- https://docs.docker.com/get-started/docker-overview/
- https://github.com/opencontainers/image-spec/blob/main/spec.md
- https://opencontainers.org/
- https://docs.docker.com/reference/dockerfile
- https://www.man7.org/linux/man-pages/man7/namespaces.7.html
- https://linux.die.net/man/1/unshare
- https://www.man7.org/linux/man-pages/man7/cgroups.7.html
- https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-registry/
- https://github.com/opencontainers/distribution-spec/blob/main/spec.md
- https://docs.docker.com/build/concepts/dockerfile/
- https://github.com/opencontainers/runtime-spec/blob/main/spec.md
- https://docs.docker.com/engine/daemon/alternative-runtimes/

# Sources


- https://github.com/containers/podman
- https://podman.io/
- Hightower, K., Burns, B., Beda, J., & Demmig, T. (2018). *Kubernetes: Eine kompakte Einführung* (1. Auflage.). dpunkt.verlag.
- https://www.redhat.com/de/topics/cloud-computing/what-is-a-hyperscaler
- https://www.ibm.com/de-de/think/topics/hyperscale
- https://cloud.google.com/learn/what-is-a-cloud-service-provider?hl=en
- https://www.redhat.com/en/topics/cloud-computing/what-are-cloud-providers
- https://www.flaticon.com/free-icon/container_7115168?term=container&page=1&position=1&origin=tag&related_id=7115168
- https://github.com/orgs/networkx/repositories

# Sources


- https://www.ibm.com/think/topics/kubernetes-history
- https://kubernetes.io/docs/setup/best-practices/cluster-large/
- https://queue.acm.org/detail.cfm?id=2898444
- https://www.cncf.io/reports/kubernetes-project-journey-report/
- https://kubernetes.io/docs/concepts/architecture/
- https://etcd.io/
- https://kubernetes.io/docs/concepts/workloads/pods/
- https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/
- Logos: https://commons.wikimedia.org/wiki/File:Google_Cloud_logo.svg, https://commons.wikimedia.org/wiki/File:IBM_Cloud_logo.png, https://commons.wikimedia.org/wiki/File:Amazon_Web_Services_Logo.svg, https://commons.wikimedia.org/wiki/File:AlibabaCloudLogo.svg, https://commons.wikimedia.org/wiki/File:Microsoft_Azure.svg